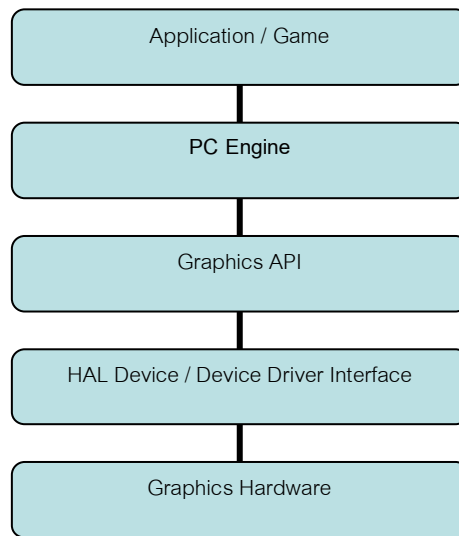


บทที่ 3

การออกแบบและทฤษฎีเพิ่มเติม

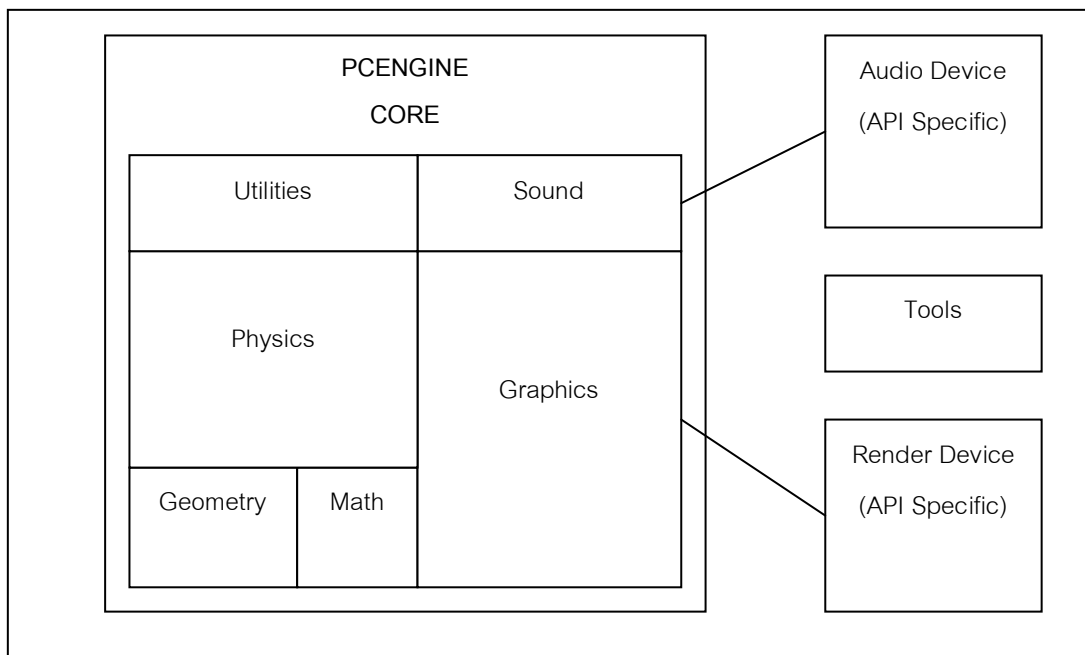
3.1 แนวทางการออกแบบCore Architecture

การทำงานของ PC Engine นั้นสามารถจะอยู่ระหว่าง Application Layer และ API Layer ดังรูปที่ 3.1



รูปที่ 3.1 PC Engine Operating Layer

จากองค์ประกอบหลักๆของ Game Engine ซึ่งกำหนดให้ส่วนกลาง (Core) เป็น API Independent จึงสามารถกำหนดลักษณะโครงสร้างคร่าวๆ ของ PC Engine ได้ดังรูปที่ 3.2



รูปที่ 3.2 โครงสร้างของ PC Engine

ส่วนประกอบต่างๆของส่วนกลางทั้งหมดจะอยู่ในข้อกำหนดของ API Independent ซึ่งจะต้องไม่มีส่วนใดอ้างถึง API ชนิดใดๆเลย ส่วนใดที่ต้องการติดต่อกับ API จะต้องออกแบบให้อยู่ในรูปของ Interface ที่สามารถติดต่อกับ API ชนิดต่างๆได้โดยสมบูรณ์ ซึ่งได้แก่ ส่วนประกอบจำพวก Render Device และ Audio Device

3.2 Mathematical Module

ส่วนนี้จะต้องช่วยอำนวยความสะดวกในการคำนวณทางคณิตศาสตร์ต่างๆ การที่จะให้ใช้งานได้ง่ายนั้นจะใช้ความสามารถของ Overloaded Operator เท่าที่จะเป็นไปได้ เพราะนอกจากผู้ใช้จะเห็นภาพของการกระทำได้ชัดเจนยิ่งขึ้นและยังง่ายต่อการเขียนสมการที่ต่อเนื่องกันอีกด้วย โดยคลาสที่สำคัญพื้นฐานได้แก่ Vector2, Vector3, Vector4, Matrix3, Matrix4 และ Quaternion

Vector2, Vector3 และ Vector4 เป็นคลาสที่ใช้ในการคำนวณ Vector Arithmetic ต่างๆมี Operation ที่จำเป็นได้แก่ +, -, *(scalar), Vector Magnitude, Normalize, Cross และ Dot Product

Matrix3 และ Matrix4 เป็นคลาสที่ใช้ในการคำนวณ Matrix Operation ต่างๆ ได้แก่ +, -, *(Matrix), *(Scalar), / (Scalar), Determinant, Adjoint, Transpose และ Inverse นอกจากนี้ Matrix3 ถูกนำไปใช้งานส่วนมากเป็น Rotation Matrix จึงต้องมีเมธอดช่วยในการสร้าง Rotation Matrix ตามแกนที่ระบุดังนี้ RotateX, RotateY, RotateZ และ RotateArbitrary

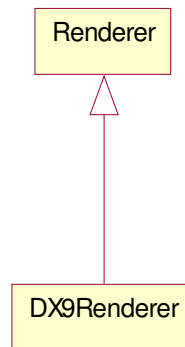
Quaternion ถูกนำไปใช้ใน Rigid body Simulation เพื่อที่จะป้องกันปัญหาที่เกิดจากความคลาดเคลื่อนของการคำนวณ Floating Point ที่จะส่งผลให้เกิด Scaling และยังใช้ใน Animation อีกด้วยเพื่อป้องกันปัญหา Gimbals' Lock ที่จะเกิดเมื่อใช้ Rotation Matrix นอกจากนี้ยังใช้ทรัพยากรน้อยกว่า Rotation Matrix และให้การ Interpolation ที่ Smooth กว่าอีกด้วย โดยมี Operation ที่สำคัญดังนี้ +, -, *(Quaternion), *(Scalar), /, Length, Dot, Normalize, Conjugate และ Inverse นอกจากนี้ยังต้องมีส่วนช่วยในการสร้าง Rotation แบบต่างๆ ตลอดจนถึงการแปลงกลับระหว่าง Quaternion และ Rotation Matrix และการทำ Interpolation ได้แก่ FromAxisAngle, ToRotationMatrix, FromRotationMatrix และ Slerp(linear spherical interpolation)

เนื่องจากส่วนนี้จะเป็นส่วนที่ถูกใช้บ่อยที่สุดในส่วนประกอบทั้งหมดของเกมเอนจิน ดังนั้นในการพัฒนาจึงถูกให้ความสำคัญทางด้านความเร็วในการดำเนินงานมากกว่าส่วนอื่นๆ โดยได้นำข้อดีของเทคโนโลยี SIMD (ในที่นี้คือ Intel SSE) มาใช้เพื่อช่วยเพิ่มความเร็วในการประมวลผลในส่วนของฟังก์ชันการทำงานที่เห็นว่าการใช้คำสั่งประเภท SIMD จะให้ความเร็วในการทำงานที่มากกว่าการใช้คำสั่งพื้นฐานของ FPU เช่น Matrix multiplication, Matrix inversion และ Vector crossing เป็นต้น

3.3 Graphics Module

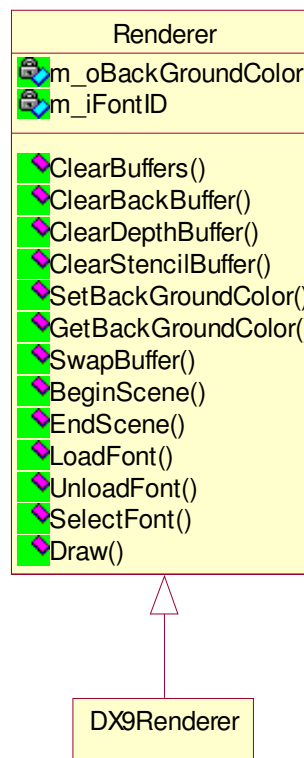
3.3.1 Basic Capability of Renderer

เพื่อให้ได้มาซึ่งความเป็น API Independent ของ Graphics Module ส่วนของ RenderDevice (Renderer) จึงเป็นเพียงแค่ Interface Class สำหรับ user ที่ติดต่อกับ Class Renderer อื่นที่ implement ด้วย API ที่เฉพาะเจาะจง



รูปที่ 3.3 คลาส DX9Renderer จะ Implement Method ต่างๆที่กำหนดโดยคลาส Renderer

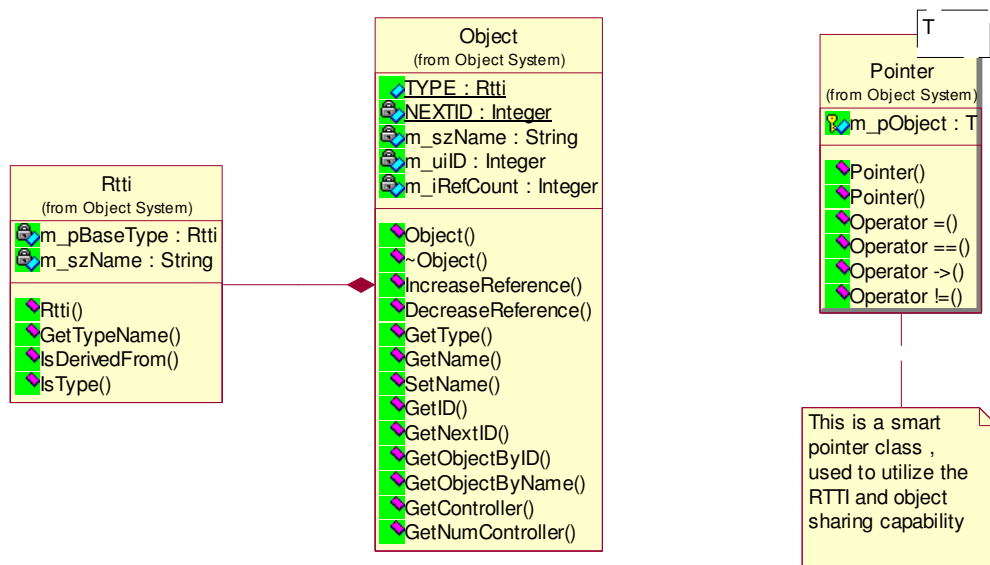
นอกจากการ Initialize และการหา Device Capability แล้วความสามารถเบื้องต้นของ คลาส Renderer ได้แก่การเคลียร์หน้าจอเป็นสีที่กำหนดได้, สามารถล้างค่าต่างๆใน Buffer ต่างๆได้, สามารถสลับ Back/Front Buffer ได้ และสามารถแสดงผลตัวอักษร (Draw) ด้วยสีต่างๆที่ตำแหน่งที่กำหนดได้ ซึ่งทั้งหมดนี้จะถูก Implement ในคลาส DX9Renderer ด้วย DirectX



รูปที่ 3.4 เมธอดพื้นฐานของคลาส Renderer

3.3.2 Object System

ใน Game Engine นั้นการจัดการกับวัตถุต่างๆมีความสำคัญมาก เนื่องจากโครงสร้างจะมีความซับซ้อน และผู้ใช้งานมีความต้องการที่จะใช้วัตถุต่างๆอยู่เสมอ การกำหนดชื่อและหมายเลขให้กับวัตถุจึงจะช่วยให้การค้นหาเป็นไปอย่างมีประสิทธิภาพยิ่งขึ้น นอกจากนี้ในบางกรณีผู้ใช้อาจต้องการทราบถึงชนิดของวัตถุเหล่านั้นในขณะนั้น (โดยเฉพาะในกรณีที่เกิดการ Casting จาก Base Class) โดยอาศัยกระบวนการ Real-time Type Information รวมไปถึงความสามารถในการใช้วัตถุต่างๆร่วมกันอย่างมีประสิทธิภาพ เช่นการนับจำนวนการถูกอ้างอิงถึงของวัตถุหนึ่งๆ (Reference counting) เพื่อที่จะสามารถทำลายวัตถุทิ้งโดยอัตโนมัติ เมื่อไม่ถูกอ้างอิงถึงอีกต่อไป (ทั้งนี้เนื่องจากพัฒนาด้วยภาษา C++ ซึ่งไม่มีระบบ Garbage collection) โดยการอ้างอิงเหล่านี้จะกระทำผ่าน Pointer ซึ่ง Pointer เหล่านี้ต้องมีความสามารถในการนับ Reference เพิ่มเติมให้กับวัตถุ

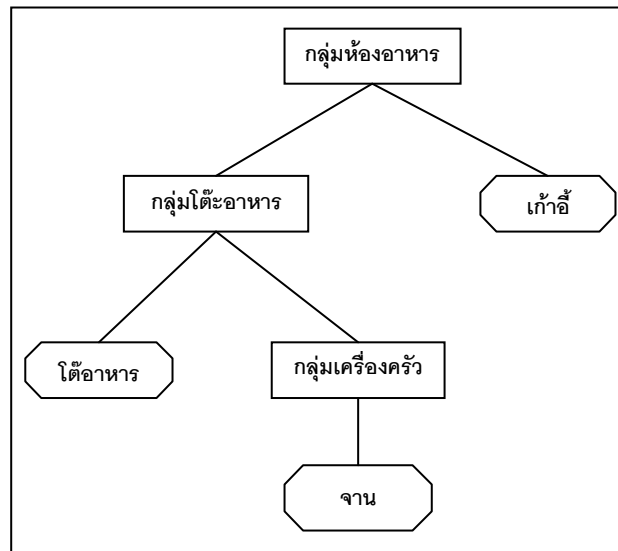


รูปที่ 3.5 Object System

ในส่วนของ Object Retrieval หรือส่วนช่วยในการค้นหา Object นั้นจะเห็นได้ว่า Object บางชนิดอาจจะบรรจุ Object ชนิดอื่นๆอยู่ จึงจะต้องทำการ Override Method ที่เกี่ยวข้องให้ทำการสืบค้นได้อย่างถูกต้อง

3.3.3 Basic Scene graph

เพื่อที่จะอธิบายฉากที่ซับซ้อนให้สะดวกและมีประสิทธิภาพยิ่งขึ้น จึงได้นำแนวคิดของ Hierarchical scene management มาใช้ โดยมีลักษณะดังภาพต่อไปนี้



รูปที่ 3.6 Scene Hierarchy

จากรูปที่ 3.6 แสดงให้เห็นถึงฉากหนึ่งในห้องอาหารซึ่งมีโต๊ะและเก้าอี้ตั้งอยู่ บนโต๊ะมีชุดของเครื่องครัวซึ่งประกอบไปด้วยจาน

เมื่อกำหนดคุณลักษณะต่างๆให้กับNodeใดๆแล้ว ลูกของNodeนั้นจะต้องมีลักษณะเดียวกันด้วย เช่นกำหนดให้กลุ่มของเครื่องครัวมีลักษณะพื้นผิว (Material) ที่เป็นมันวาว ไม่ว่าจะเพิ่มซ้อนหรือซ้อนเข้ามาก็จะมีลักษณะมันวาวเช่นเดียวกันกับจาน หรือแม้แต่การกำหนดแสงให้กับกลุ่มห้องอาหาร วัตถุทั้งหมดในห้องอาหารก็จะได้รับผลจากแสงนั้นด้วยเป็นต้น โดยตัวอย่างที่เห็นได้อย่างชัดเจนที่สุดคือการคำนวณเกี่ยวกับ Transformation และ Bounding Volume ของแต่ละNode

เมื่อพิจารณาในส่วนของการ Transformation (World and local transformation) สามารถอธิบายได้ดังต่อไปนี้ ซึ่ง World Transformation จะถูกนำไปใช้กับ Graphic API ในการกำหนดตำแหน่งของวัตถุ

$$\text{World(Room)} = \text{Local(Room)}$$

$$\begin{aligned}\text{World(Chair)} &= \text{World(Room)} \cdot \text{Local(Chair)} \\ &= \text{Local(Room)} \cdot \text{Local(Chair)}\end{aligned}$$

$$\begin{aligned}\text{World(Table Group)} &= \text{World(Room)} \cdot \text{Local(Table Group)} \\ &= \text{Local(Room)} \cdot \text{Local(Table Group)}\end{aligned}$$

$$\begin{aligned}\text{World(Table)} &= \text{World(Table Group)} \cdot \text{Local(Table)} \\ &= \text{Local(Room)} \cdot \text{Local(Table Group)} \cdot \text{Local(Table)}\end{aligned}$$

$$\begin{aligned}
 \text{World(Utensil Group)} &= \text{World(Table Group)} \cdot \text{Local(Utensil Group)} \\
 &= \text{Local(Room)} \cdot \text{Local(Table Group)} \cdot \text{Local(Utensil Group)} \\
 \text{World(Disc)} &= \text{World(Utensil Group)} \cdot \text{Local(Disc)} \\
 &= \text{Local(Room)} \cdot \text{Local(Table Group)} \cdot \text{Local(Utensil Group)} \cdot \text{Local(Disc)}
 \end{aligned}$$

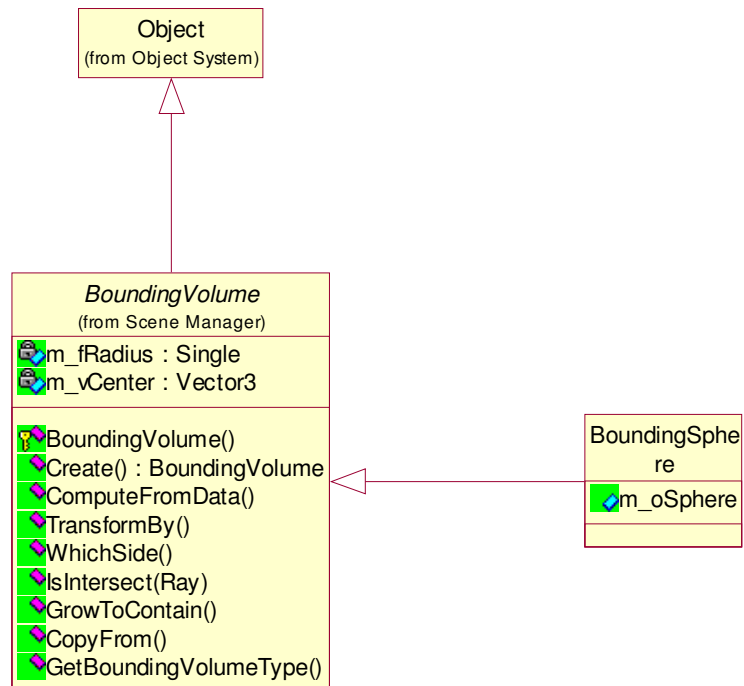
จะเห็นได้ว่าสามารถจัดการการเคลื่อนย้ายวัตถุที่มีความสัมพันธ์กันได้โดยง่าย เช่นการย้ายกลุ่มของโต๊ะจะทำให้ทุกสิ่งที่อยู่บนโต๊ะถูกเคลื่อนย้ายไปด้วยเป็นต้น

นอกจากการใช้ประโยชน์ในเรื่องของความสามารถในการอธิบายฉากต่างๆที่มีความซับซ้อนได้แล้ว Scene graph ยังสามารถเพิ่มประสิทธิภาพให้กับเอนจิน โดยจะสามารถเห็นได้ชัดเมื่อมีวัตถุจำนวนมากอยู่ในฉาก การส่งวัตถุทั้งหมดไปให้กับการ์ดแสดงผลจะมีผลกระทบกับประสิทธิภาพการทำงานได้ (ถึงแม้ว่าการ์ดแสดงผลจะมีกระบวนการ Culling อยู่บน Pipeline ก็ตาม แต่ก็ยังจะต้องเสียเวลาทำการ Transform กับทุกๆ Vertices ของวัตถุก่อนอยู่ดีจึงจะทราบว่าวัตถุนั้นอยู่นอกบริเวณที่สามารถมองเห็นได้) การทำ Visibility test นั้นจะสามารถทำได้เมื่อเราใช้ประโยชน์จาก Bounding Volume ของแต่ละNode เราจะสามารถทราบได้ว่าวัตถุที่อยู่ภายในห้องอาหารทั้งหมดไม่สามารถมองเห็นได้จากการตรวจสอบเพียงครั้งเดียวในNodeกลุ่มห้องอาหาร โดยวิธีการตรวจสอบจะกล่าวถึงในส่วนของ Frustum Culling การออกแบบเชิงวัตถุของ Scene graph จะถูกกำหนดตามคุณสมบัติต่างๆตามที่ได้กล่าวมาดังต่อไปนี้

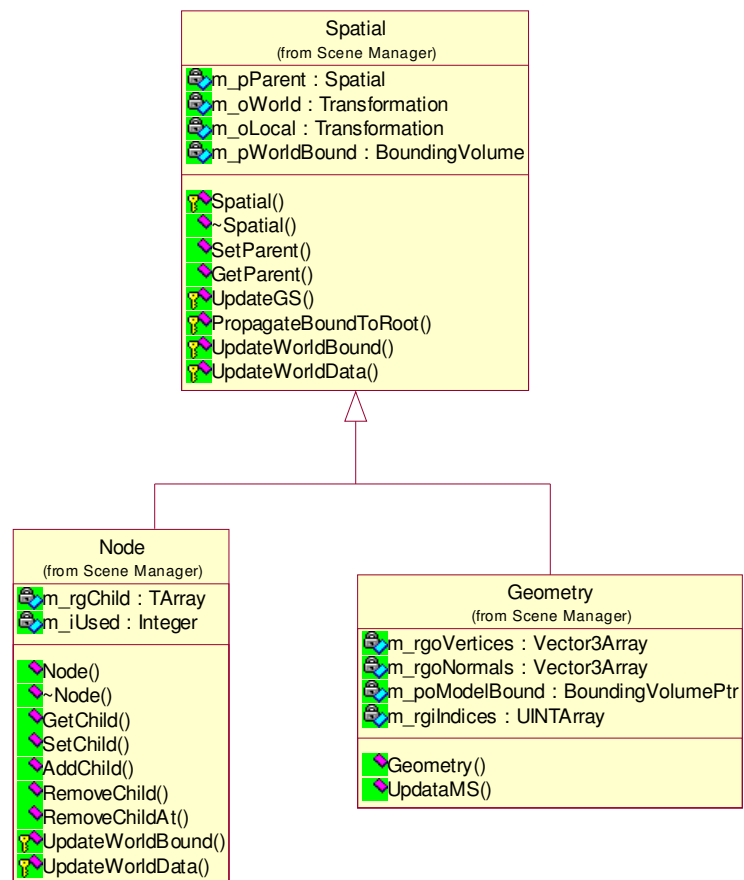
Spatial มีคุณสมบัติพื้นฐานได้แก่การเปลี่ยนจาก Local transformation -> World transformation และคำนวณจาก World bounding volume จาก Local Bounding Volume

Node จะทำหน้าที่กระจายกระบวนการ Update State ต่างๆไปยัง children ทุกตัว (เฉพาะวัตถุนั้นเท่านั้นที่สามารถมีลูกได้)

Geometry เป็นคลาสสำหรับวัตถุซึ่งจะปรากฏที่ leaf node ของ tree เท่านั้น โดยจะเก็บข้อมูลเกี่ยวกับ Local bounding volume (Model bound) และข้อมูลเกี่ยวกับวัตถุได้แก่ Vertices, Indices และ Normal โดยการคำนวณขนาดและรูปร่าง Model Bound สามารถเลือกทำได้หลายรูปแบบและวิธี ในที่นี้เลือกใช้ Bounding Sphere โดยแต่ละชนิดก็จะใช้วิธีที่แตกต่างกันออกไปในการคำนวณ นอกจากนี้ Bounding Volume ยังต้องสามารถขยายตัวเองให้ครอบคลุม Bounding Volume ในระดับต่ำกว่าและย้อนกลับไปยัง Root Node ได้ด้วย และยังสามารถตรวจสอบได้ว่าอยู่ด้านใดของ Plane เพื่อใช้ในการทำ Frustum Culling และสามารถตรวจจับการชนกันกับ Ray สำหรับการทำให้ Picking



រូបភាព 3.7 BoundingVolume និង BoundingSphere



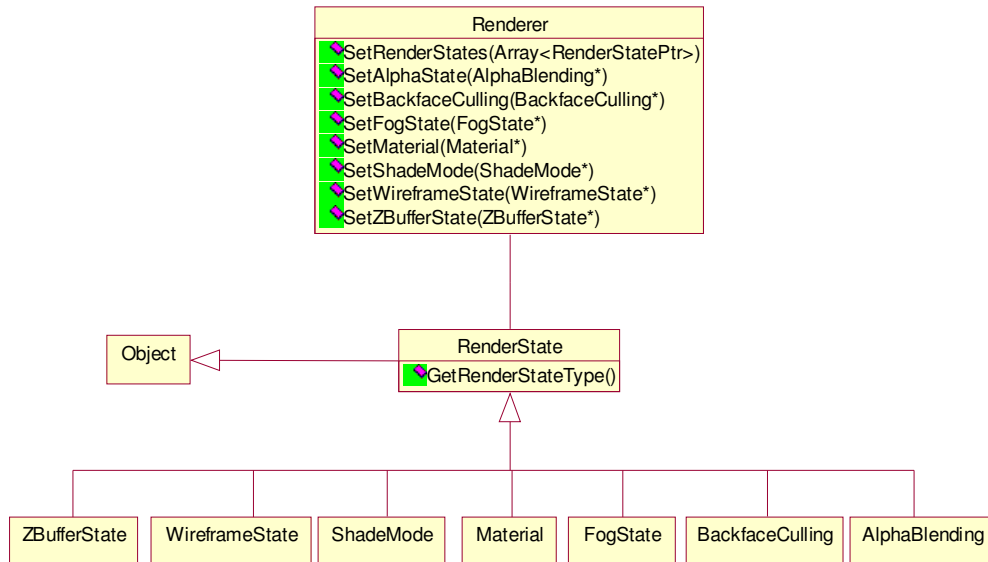
រូបភាព 3.8 Basic Scene Graph



ในการแสดงผลภาพสามมิติ (Draw Primitive) การแสดงผลของ Graphics API ส่วนมากจะใช้รูปแบบของ State Machine การกำหนดใช้ Render State ต่างๆก็เช่นเดียวกัน โดย Render State ประกอบไปด้วยส่วนต่างๆได้แก่ การแสดงผลแบบ Wireframe, การทำ Back Face Culling, การ

กำหนด Shade mode, การกำหนดค่า Alpha Blending, การกำหนดค่า Fog Effect, การกำหนด Material และการกำหนดสถานะของ Z-Buffer

ส่วนของคลาส Renderer จะต้องสามารถกำหนด Render State ต่างๆได้อย่างสะดวก เนื่องจาก Render State ชนิดต่างๆมีรายละเอียดและวิธีการกำหนดค่า ต่างๆกันไปในแต่ละ API



รูปที่ 3.11 โครงสร้างการทำงานของ Render State

เมธอดในการติดตั้ง Render State ต่างๆเช่น SetAlphaState, SetMaterial จะ Implement ไว้ใน Derived Renderer Class เนื่องจากแต่ละ API มีวิธีที่แตกต่างกันออกไป

3.3.4.1 Alpha Blending

กระบวนการ Alpha Blending ใน Render State คือการกำหนดค่าสำหรับการทำ Frame Buffer Alpha Blending ซึ่งจะใช้ค่า Alpha ที่รับมาจาก Vertex Color หรือ Material หรือ Texture ซึ่งจะกำหนดค่า Alpha (Transparency Value) สำหรับในแต่ละวัตถุ มาใส่ใน Frame Buffer

เมื่อทำ Alpha Blending สีสองสีจะถูกรวมเข้าด้วยกัน คือ Source Color และ Destination Color โดย Source Color มาจากวัตถุที่โปร่งแสง และ Destination Color มาจากค่าที่อยู่บน Frame Buffer อยู่แล้วซึ่งมาจากผลของการ Render วัตถุอื่นๆก่อนหน้านี้ โดยการควบคุมกระบวนการเป็นไปตามสูตรต่อไปนี้

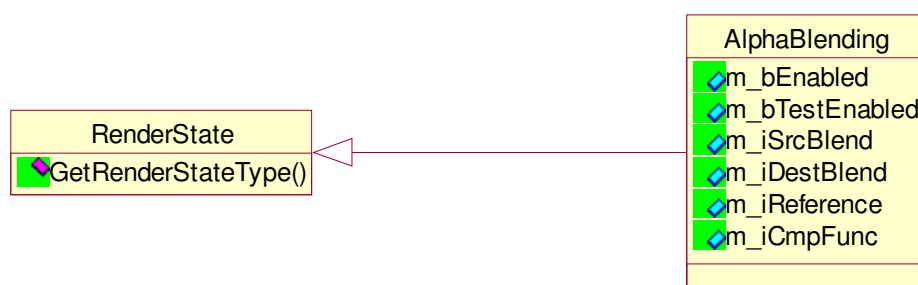
$$\text{FinalColor} = \text{ObjColor} * \text{SrcBlendFactor} + \text{PixelColor} * \text{DestBlendFactor} \quad (3.1)$$

ObjectColor คือสีของวัตถุที่กำลังจะถูก Render ที่ตำแหน่งนั้น ส่วน PixelColor คือสีปัจจุบันที่ปรากฏอยู่บน Frame Buffer ที่ตำแหน่งนั้น โดยค่า Blend Factor ต่างๆสามารถเป็นไปตามตารางที่ 3.1

Blend Factor	รายละเอียด
ZERO	(0, 0, 0, 0)
ONE	(1, 1, 1, 1)
SRCCOLOR	(Rs, Gs, Bs, As)
INVSRCOLOR	(1-Rs, 1-Gs, 1-Bs, 1-As)
SRCALPHA	(As, As, As, As)
INVSRCALPHA	(1-As, 1-As, 1-As, 1-As)
DESTALPHA	(Ad, Ad, Ad, Ad)
INVDESTALPHA	(1-Ad, 1-Ad, 1-Ad, 1-Ad)
DESTCOLOR	(Rd, Gd, Bd, Ad)
INVDESTCOLOR	1-Rd, 1-Gd, 1-Bd, 1-Ad)
SRCALPHASAT	(f, f, f, 1); f = min(As, 1-Ad)

ตารางที่ 3.1 รายละเอียดของ Blend Factor

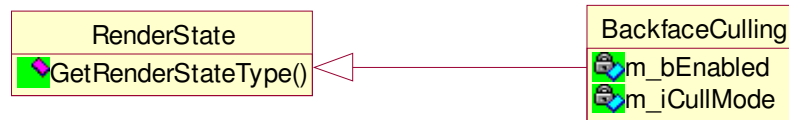
นอกจาก AlphaBlending จำเป็นต้องเก็บค่าต่างๆ ที่สมการต้องการได้ ซึ่งได้แก่ Source และ Destination Blend Factor แล้วยังต้องมีความสามารถในการทดสอบว่าควรจะทำการเขียนค่า Alpha ลงบน Frame Buffer หรือไม่ (Alpha Testing) โดยทำการเทียบค่า Alpha บน Frame Buffer กับค่า Reference ค่าหนึ่งด้วย Condition ต่างๆ ได้แก่ Never, Always, Less, Less Equal, Equal, Greater, Greater Equal และ Not Equal



รูปที่ 3.12 AlphaBlending Class

3.3.4.2 Back-face Culling

BackfaceCulling เป็นการตัด Polygon ที่มองไม่เห็นออกเนื่องจากหันหลังให้กับกล้อง โดยจะดูจากค่า Normal และทิศทางของกล้องเป็นหลักซึ่งสามารถกำหนดทิศได้ตามลักษณะ Index ของการวาดว่าเป็นการวาดทวนเข็มนาฬิกา หรือตามเข็มนาฬิกา (Cull ModeJ)



รูปที่ 3.13 BackfaceCulling Class

3.3.4.3 Fog State

Graphic Hardware ในปัจจุบันสนับสนุนการทำงานของ Pixel Based Fog ซึ่งจะกระทำในขั้นตอนของ Rasterization โดยจะทำการ Blend ค่าสีของ Fog เข้ากับค่าสีบน Frame Buffer ดังสมการต่อไปนี้

$$I = \text{FogFunction}(\text{Distance})$$

$$\text{FinalColor} = I \times \text{CurrentColor} + (1-I) \times \text{FogColor} \quad (3.2)$$

Fog Function ต่างๆ ได้แก่ Linear, Exponential และ Exponential Squared โดยมีสูตรดังต่อไปนี้

//Linear Fog Function

$$I = \frac{\text{fogend} - \text{distance}}{\text{fogend} - \text{fogstart}}$$

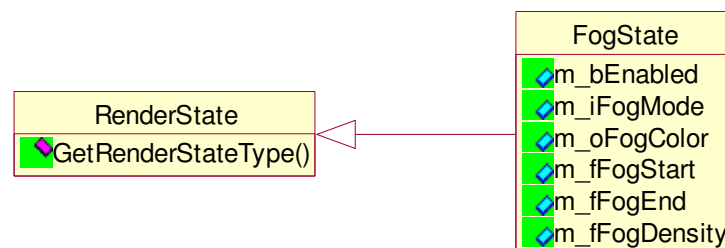
//Exponential Fog Function

$$I = \frac{1}{e^{\text{distance} \times \text{density}}}$$

//Exponential Squared Fog Function

$$I = \frac{1}{e^{(\text{distance} \times \text{density})^2}}$$

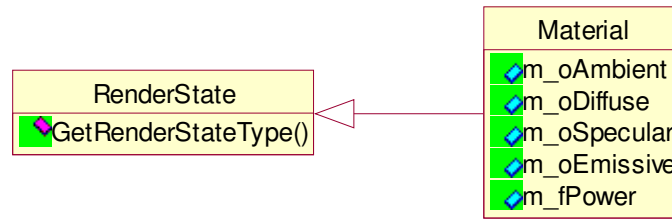
ดังนั้นคลาส Fog จะต้องเก็บข้อมูลได้แก่ Fog Mode (Function), Fog Color, Fog Start, Fog End และ Fog Density



รูปที่ 3.14 FogState Class

3.3.4.4 Material

ความรู้เบื้องต้นเกี่ยวกับ Material ได้อธิบายไว้แล้วในบทที่ 2 การออกแบบจะต้องสามารถเก็บข้อมูลทั้งหมดที่เป็นคุณสมบัติของ Material ไว้ให้ได้เพื่อให้ Derived Renderer จะสามารถนำไปใช้ได้ถูกต้อง



รูปที่ 3.15 Material Class

3.3.4.5 Shade Mode

Shade mode ใช้ระบุถึงความเข้มของแสงที่จุดใดจุดบน Face ของ Polygon โดยส่วนมาก Graphics API จะสนับสนุนการทำงานสองแบบ คือ Flat Shading และ Gouraud Shading

Flat Shading จะใช้สีของ Material หรือสีของ Vertex แรกของ Face สำหรับทั้ง Face ส่วน Gouraud Shading จะใช้ Vertex Normal และค่าคุณสมบัติของแสงมาคำนวณสีของแต่ละ Vertex จากนั้นทำการ Interpolate สีด้วย Linear Interpolation จะทำให้ดูนุ่มนวลกว่า

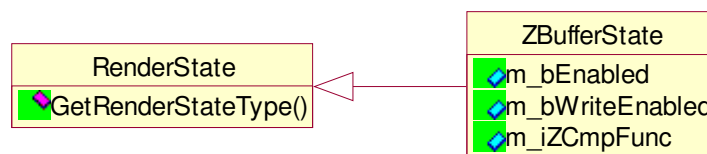
การออกแบบคลาส Shade Mode จะต้องเก็บคุณลักษณะที่สำคัญเพียงอย่างเดียวคือ Shade Mode เท่านั้น

3.3.4.6 Wireframe State

Wireframe State ใช้แสดงถึงวิธีในการวาดภาพว่าจะทำการ Fill Polygon หรือให้ทำ Line Drawing เพียงอย่างเดียว คุณสมบัติของคลาสที่สำคัญจึงมีเพียงแค่สถานะเปิดหรือปิดเท่านั้น

3.3.4.7 Z-Buffer State

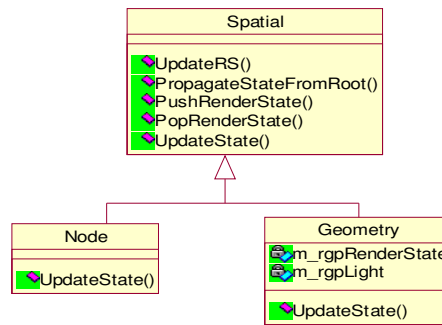
ในที่นี้หมายถึงเพียงแค่ส่วนของ Depth Buffer ซึ่งได้อธิบายไปแล้วในบทที่ 2 โดยคุณสมบัติของ Class ที่สำคัญได้แก่ Z-Compare Function ที่ใช้ในการทดสอบเพื่อเขียนลงบน Frame Buffer ได้แก่ Equal, Less, Less Equal, Greater, Greater Equal และ Not Equal, สถานะภาพใช้งาน และสถานะภาพเขียนค่า Z ลงบน Buffer เท่านั้น



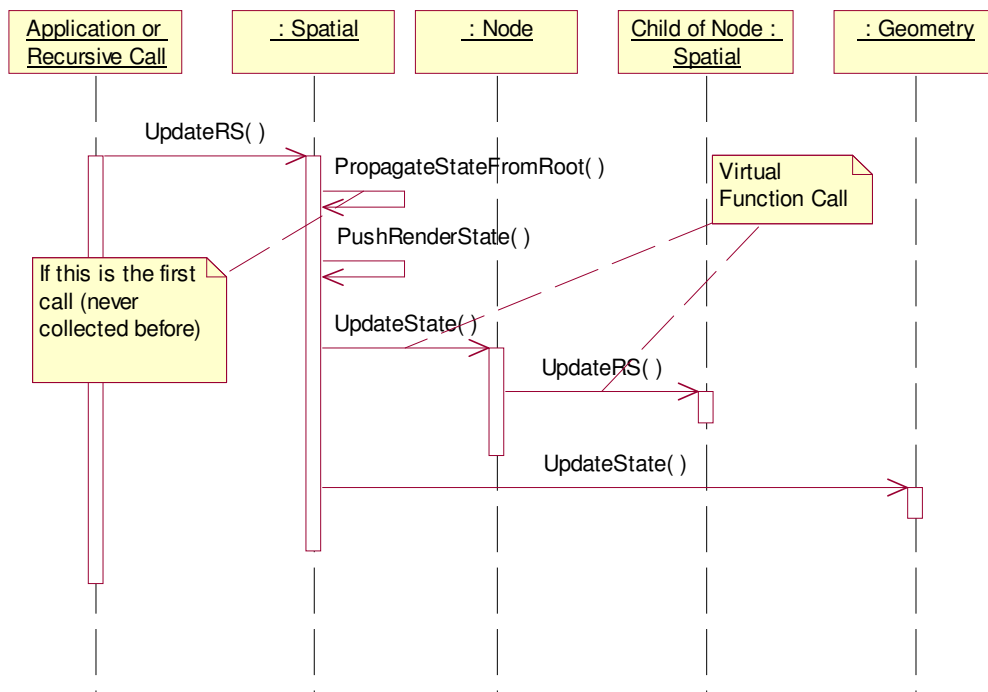
รูปที่ 3.16 ZBufferState Class

3.3.4.8 Update Render State ใน Scene graph

การ Update Render State ใน Scene graph สามารถทำได้โดยไล่สะสม State มาจาก Root Node โดยจะรวมไปถึงสะสม Light ที่ได้ติดตั้งไว้ตั้งแต่ Root Node ลงมาด้วย โดยเมื่อถึง Geometry จะต้องนำ State ล่าสุดที่ผ่านมามาใส่ให้กับ Geometry เพื่อนำไปใช้ในการ Render



รูปที่ 3.17 Update Render State



รูปที่ 3.18 Sequence Diagram: Update Render State

3.3.5 Effect & Texture Mapping

PC Engine สนับสนุนการใช้ Vertex Color และ Multi-texture ผ่านทางคลาส Effect (แต่เดิมควรจะชื่อ Skin เพียงแต่จะสร้างความสับสนให้กับผู้ใช้ ระหว่าง Skin ใน Skeletal Animation) โดยคุณสมบัติของคลาส Effect จะประกอบไปด้วย Vertex Color, Textures และ UVs (Texture Coordinates) ส่วนของ Vertex Color มีความตรงไปตรงมาคือ Color ของแต่ละ Vertex แต่ในส่วน of คลาส Texture นอกจากคุณสมบัติพื้นฐานอย่างเช่น Image ที่ใช้ Filter ที่ใช้ และ UV Mode (Addressing Mode) แล้วจะต้องสามารถให้ข้อมูลในการทำ Multi-texture ได้ด้วย Apply Mode แบบต่างๆ ได้แก่ Replace, Modulate, Decal, Blend, Add และ Combine ซึ่งเป็นตัวกำหนด Operation ที่ Texture Stage ต่างๆ (กรณีใช้ Multi-texture) ได้อย่างมีประสิทธิภาพ โดยในกรณีของ Combine Apply Mode จะมีดังต่อไปนี้

Apply Mode	คำอธิบาย
REPLACE	FinalColor = TextureColor, FinalAlpha = TextureAlpha
DECAL	FinalColor = (1-TextureAlpha) x CurrentColor + TextureAlpha x TextureColor, FinalAlpha = CurrentAlpha
MODULATE	FinalColor = TextureColor x CurrentColor, FinalAlpha = TextureAlpha x CurrentAlpha
BLEND	FinalColor = (1 - TextureAlpha) x CurrentColor + TextureAlpha x TextureColor, FinalAlpha = TextureAlpha x CurrentAlpha
ADD	FinalColor = CurrentColor + TextureColor, FinalAlpha = CurrentAlpha x TextureAlpha
COMBINE	Customizable Textures Blending

ตารางที่ 3.2 Texture Apply Mode

ผู้ใช้สามารถกำหนดการใช้ Texture ได้เองอย่างอิสระทั้งในส่วนของ Color และ Alpha Channel ในกรณีที่กำหนด Apply Mode เป็น COMBINE ซึ่งจะขึ้นกับ Combine Function ต่างๆ ดังนี้

Combine Function	สมการ
REPLACE	$v0$
MODULATE	$v0 \times v1$
ADD	$v0 + v1$
ADD_SIGNED	$v0 + v1 - \frac{1}{2}$
SUBSTRACT	$v0 - v1$
INTERPOLATE	$v0 \times v2 + v1 \times (1 - v2)$

ตารางที่ 3.3 Combine Function

โดย v_i คือ Operand ซึ่งจะกำหนดโดย Combine Source และ Combine Operation ต่างๆ ด้วยกันดังนี้

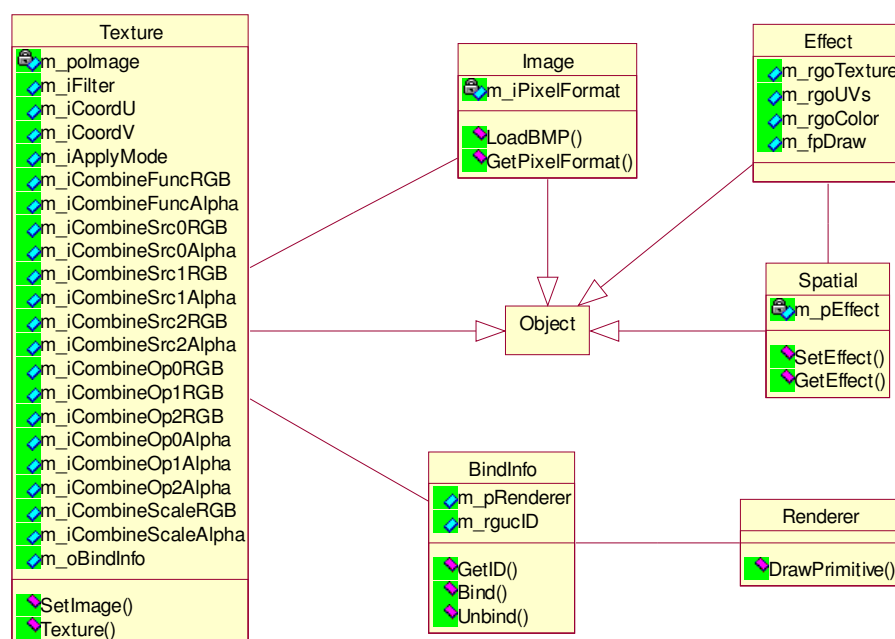
Source/Operation	Source Color	1- Source Color	Source Alpha	1 – Source Alpha
TEXTURE	TextureColor	1- TexColor	TextureAlpha	1- TexAlpha
PRIMARY_COLOR	CurrentColor	1- CurrColor	CurrentAlpha	1- CurrAlpha
CONSTANT	ConstantColor	1- ConstColor	ConstantAlpha	1- ConstAlpha
PREVIOUS	PreviousColor	1- PrevColor	PreviousAlpha	1- PrevAlpha

ตารางที่ 3.4 Combine Function Operand

จากตารางที่ 3.3 จะเห็นได้ว่าข้อมูลด้านคุณสมบัติที่ใช้ในการทำ Texture Combine สามารถประกอบไปด้วยจำนวนสูงสุดถึง 3 ชุด คือ v0 v1 และ v2 ซึ่งแต่ละชุดประกอบไปด้วย RGB และ Alpha Channel ทั้ง Source และ Operation นอกจากนี้จะต้องสามารถทำการ Scale ผลลัพธ์ได้ด้วยเป็นจำนวนเท่าได้แก่ 1 2 และ 4 เท่า

นอกจากคุณสมบัติดังกล่าวมาทั้งหมดของ Class Effect แล้วจะเห็นได้ว่าในบางกรณี Drawing Function ที่ใช้งานอยู่อาจจะไม่ตรงความต้องการของผู้ใช้ เช่นการใช้ Texture ในทางอื่นที่นอกเหนือจากปกติ หรือการทำ Multi-pass Rendering ซึ่งต้องการผลจากการวาด Node อื่นก่อน (Global Effect) คลาส Effect จึงต้องสามารถกำหนดให้ผู้ใช้สามารถใช้ Drawing Function ของตัวเองได้ด้วย อย่างไรก็ตามโดยปกติแล้วจะเป็น Drawing Function พื้นฐานของ PC Engine

การที่ Texture ได้ถูกเรียกใช้โดย Renderer แล้วครั้งหนึ่งแสดงว่า Texture นั้นถูกจัดการและเก็บบน Video Memory หรือส่วนอื่นๆที่จัดการโดย Graphics API แล้วไม่มีความจำเป็นที่จะสร้าง Texture นั้นซ้ำอีกครั้งจึงทำการ Bind Texture Object ที่ใช้บน API นั้นเข้ากับ Texture Object ของ PC Engine

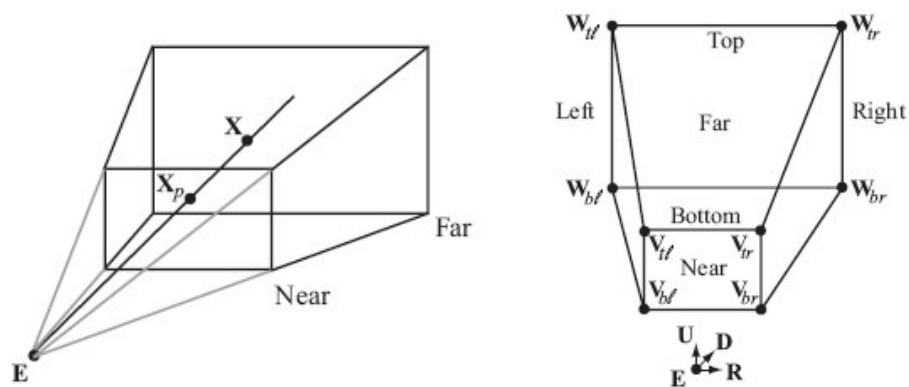


รูปที่ 3.19 Effect Class

3.3.6 Scene Graph Rendering

การแสดงผลต่างๆด้วย Scene graph นั้นเริ่มต้นจากคลาส Camera ซึ่งจะถูกติดตั้งให้กับ Render Device โดยคุณลักษณะของ Camera ประกอบไปด้วย View Frustum, Viewport, Frame และมีหน้าที่ทำ Visibility testing (Culling), สร้าง Ray สำหรับนำไปใช้ในการทำPicking ด้วย

การที่ Camera มีคุณสมบัติเช่นเดียวกับ Spatial คือมีตำแหน่ง Location และ Orientation ทำให้เราสามารถสืบทอดคุณสมบัติมาจากคลาส Spatial ได้ การทำเช่นนี้ยังทำให้ Camera เป็นส่วนหนึ่งของฉากนั้นๆด้วยเช่นกันโดยเชื่อมกล่องวงจรปิดที่ติดอยู่ในห้องแต่ละห้องเป็นต้น โดยส่วนของ Frame ของกล่องซึ่งก็คือ Location, Direction, Up, Right สามารถใช้ Location, Rotation Matrix ในส่วนของ Transformation แทนได้ให้ใช้หน่วยความจำอย่างมีประสิทธิภาพ



รูปที่ 3.20 Standard Camera Model

รูปที่ 3.20 แสดงถึง Camera Frustum และ Plane ที่เกี่ยวข้องซึ่งตำแหน่งที่ต้องคำนวณจาก Frustum Parameter ทั่วไปเช่น Near Far Left Right Top Bottom ได้แก่ตำแหน่ง Vtl Vtr Vbl และ Vbr เป็น Vertex ของ Near Plane ส่วนของ Far Plane ก็จะใช้ลักษณะเดียวกันได้แก่ Wtl Wtr Wbl และ Wbr

$$Vbl = E + dminD + uminU + rminR$$

$$Vtl = E + dminD + umaxU + rminR$$

$$Vbr = E + dminD + uminU + rmaxR$$

$$Vtr = E + dminD + umaxU + rmaxR$$

$$Wbl = E + dmax/dmin (dminD + uminU + rminR)$$

$$Wtl = E + dmax/dmin (dminD + umaxU + rminR)$$

$$Wbr = E + dmax/dmin (dminD + uminU + rmaxR)$$

$$Wtr = E + dmax/dmin (dminD + umaxU + rmaxR) \quad (3.3)$$

ส่วนของการทำ Frustum Culling จะเป็นการทดสอบกับแต่ละ Frustum Plane ที่มี Normal ที่เข้าด้านใน ถ้า Bounding Volume ใดอยู่หลัง Plane ก็จะมองไม่เห็น โดยสมการของ Plane สามารถหาได้จากสูตรทั่วไปของ Plane ซึ่งจุด X ใดๆบน Near Plane คือ $E + dminD$

$$D \cdot X = D \cdot (E + dminD) = D \cdot E + dmin \quad (3.4)$$

ในส่วนของ Far Plane ก็เช่นเดียวกัน X คือ $E + d_{\max}D$

$$-D \cdot X = -D \cdot (E + d_{\max}D) = -D \cdot E - d_{\max} \quad (3.5)$$

ส่วนของ Left Plane จะประกอบไปด้วยจุด 3 จุดคือ E, Vtl และ Vbl ดังนั้น Normal Vector คือ

$$\begin{aligned} (Vbl - E) \times (Vtl - E) &= (d_{\min}D + u_{\min}U + r_{\min}R) \times (d_{\min}D + u_{\max}U + r_{\min}R) \\ &= (d_{\min}D + r_{\min}R) \times (u_{\max}U) + (u_{\min}U) \times (d_{\min}D + r_{\min}R) \\ &= (d_{\min}D + r_{\min}R) \times ((u_{\max} - u_{\min})U) \\ &= (u_{\max} - u_{\min})(d_{\min}D \times U + r_{\min}R \times U) \\ &= (u_{\max} - u_{\min})(d_{\min}R - r_{\min}D) \end{aligned} \quad (3.6)$$

ดังนั้นเวกเตอร์ Normal หนึ่งหน่วยของ Left Plane และ สมการของ Left Plane คือ

$$N_l = \frac{d_{\min}R + r_{\min}D}{\sqrt{d_{\min}^2 + r_{\min}^2}}, N_r \cdot (X - E) = 0 \quad (3.7)$$

ในทำนองเดียวกันก็จะสามารถหาสมการของ Right Bottom และ Top Plane ได้ดังนี้

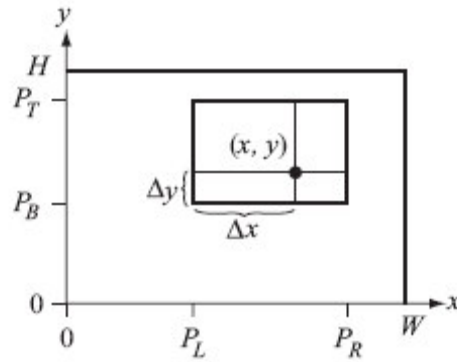
$$N_r = \frac{-d_{\min}R + r_{\max}D}{\sqrt{d_{\min}^2 + r_{\max}^2}}, N_r \cdot (X - E) = 0 \quad (3.8)$$

$$N_b = \frac{d_{\min}U + u_{\min}U}{\sqrt{d_{\min}^2 + u_{\min}^2}}, N_b \cdot (X - E) = 0 \quad (3.9)$$

$$N_t = \frac{-d_{\min}R + u_{\max}D}{\sqrt{d_{\min}^2 + u_{\max}^2}}, N_t \cdot (X - E) = 0 \quad (3.10)$$

สมการ Plane ต่างๆจะถูกเก็บไว้เฉพาะส่วนของ Coefficient เพื่อนำไปใช้คำนวณหา World Plane ที่ใช้ในการ Cull จริงๆ (World Plane เก็บในรูปแบบของ Plane Object) ส่วนของ Viewport Parameters จะต้องถูก Normalize เนื่องจากว่ายังไม่ทราบความกว้างและความสูงของส่วนที่ใช้แสดงผลจริง (ขึ้นกับ Renderer) คุณสมบัติที่สำคัญได้แก่ Left Right Top และ Bottom โดยการเปลี่ยนแปลงทั้งหมดที่เกิดขึ้นกับ Camera จะต้องแจ้งให้ Renderer ทราบด้วยเพื่อที่จะส่งต่อไปให้กับ API ไปจัดการ

นอกจากการทำ Frustum Culling แล้ว Camera ยังสนับสนุนการทำ Picking ด้วยการช่วยสร้าง Picking Ray จากตำแหน่งของ Camera Ffp มีทิศเดียวกันกับบริเวณที่กำหนดบน Viewport ดังนี้



รูปที่ 3.21 กรณียของ Non-fullscreen viewport

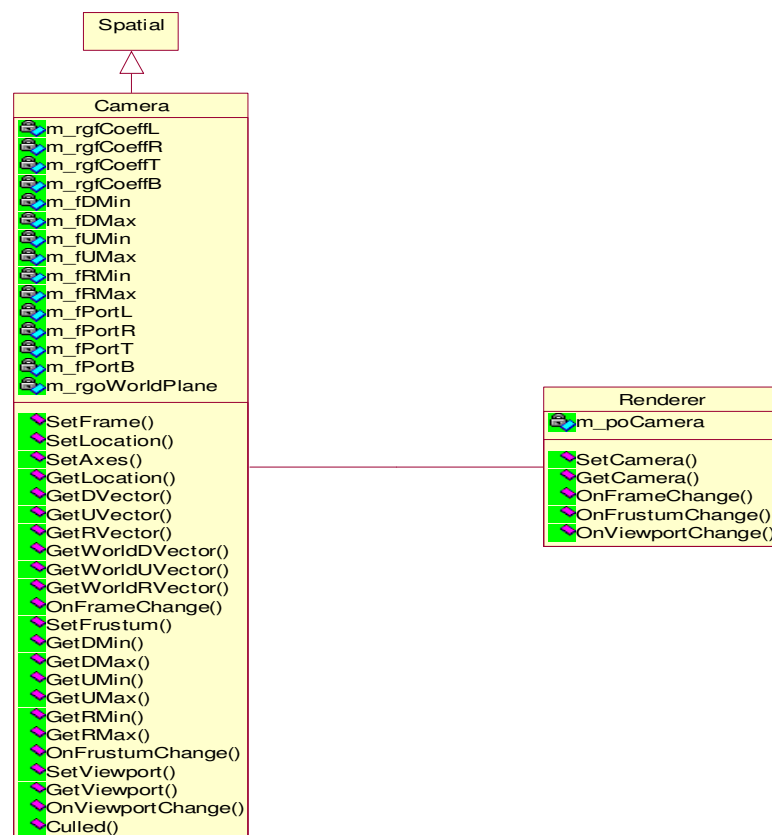
เริ่มจาก Normalize ตำแหน่งที่กำหนดบน Screen Coordinate (x, y) โดยการ Inverse ค่า y ด้วยเนื่องจาก Screen Coordinate ถือว่า y = 0 อยู่ด้านบน

$$x' = x / (W - 1), y' = H - 1 - y / (H - 1) \quad (3.11)$$

$$\Delta x = \frac{x' - P_L}{P_R - P_L}, \Delta y = \frac{y' - P_B}{P_T - P_B} \quad (3.12)$$

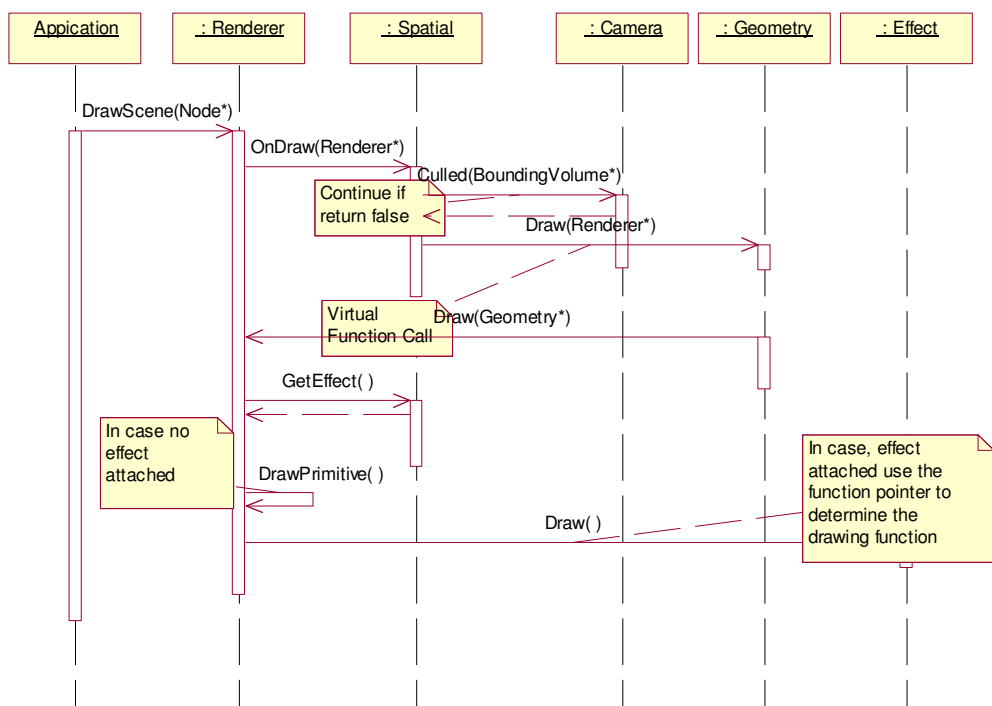
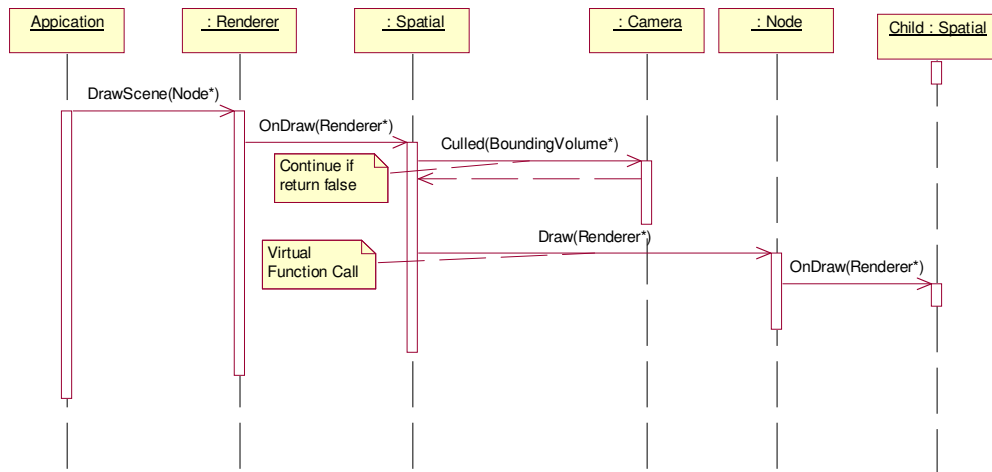
$$\text{Pick Ray} = E + tU$$

$$U = d \min D + ((1 - \Delta x)r \min + \Delta x r \max)R + ((1 - \Delta y)u \min + \Delta y u \max)U \quad (3.13)$$



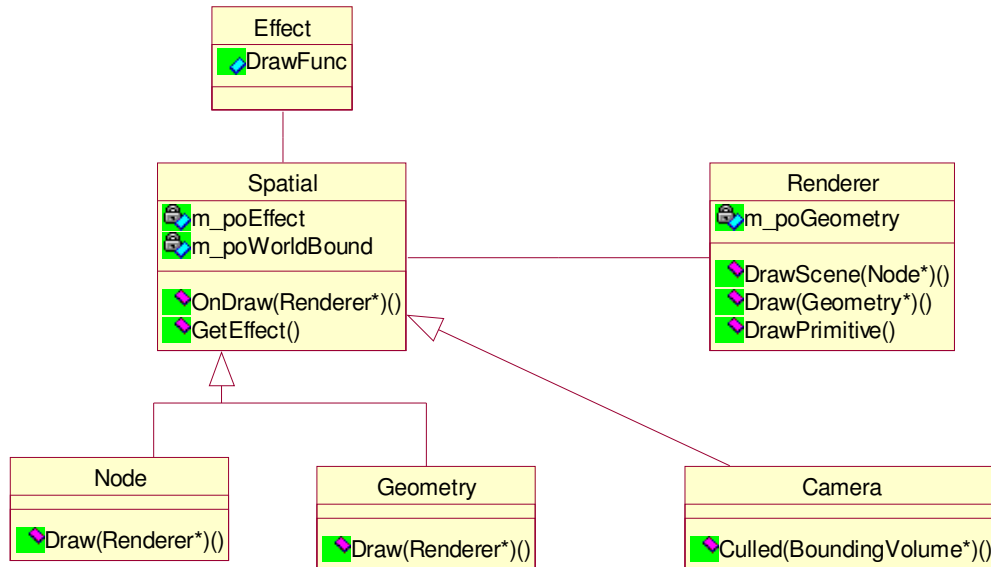
รูปที่ 3.22 Camera Class

การทำ Render Scene graph นั้นจะเป็นการทำตามลำดับการเดินทางแบบ Depth first Traversal การวาดแบบ Single pass มีลำดับขั้นตอนดังต่อไปนี้



รูปที่ 3.23 Sequence Diagram: Single Pass Drawing

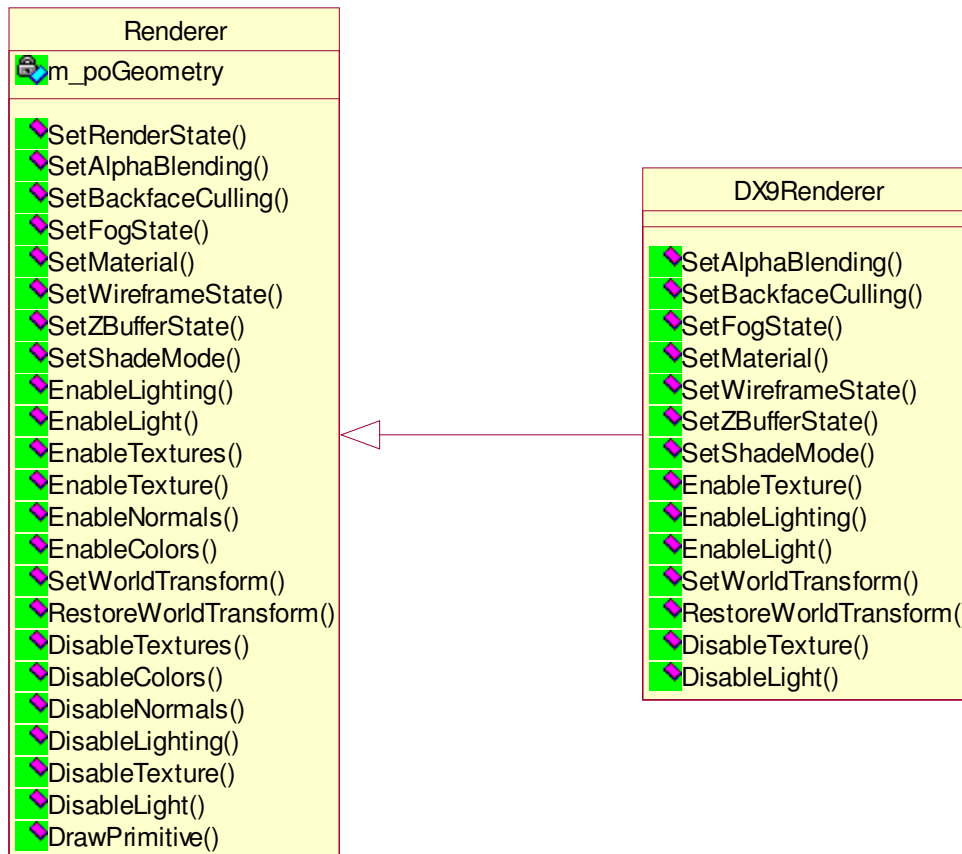
ในบางกรณีที่ต้องการสร้าง Effect ที่อาศัย Multi-pass Rendering (Global Effect) Effect Object จะต้องใช้ Drawing Function พิเศษที่กำหนดขึ้นเอง อย่างไรก็ตาม Effect ประเภทนี้จะติดตั้งอยู่บน Node ไม่ใช่ Geometry ดังนั้นในช่วงของการเรียก Node::Draw(Renderer*) จะแบ่งเป็นสองกรณีคือมี Effect และไม่มีในกรณีที่มี Effect ก็จะต้องเรียก Render::Draw(Node*) ก่อนเพื่อทำการ Draw Effect ที่ติดตั้งไว้ด้วย Drawing Function พิเศษ ซึ่งอาจรวมไปถึงการ Draw ลูกๆ ด้วยก็ได้



รูปที่ 3.24 Drawing Related Class Diagram

ในเมธอด `DrawPrimitive` จะดึงเอาคุณสมบัติต่างๆ ที่ติดมากับ `Geometry` เพื่อนำไปกำหนดให้กับ `Derived Renderer` มีขั้นตอนต่างๆดังนี้

- Set Render States
- Enable Lights
- Enable Vertices
- Enable Vertex Normals
- Enable Vertex Colors
- Enable Textures
- Apply World Transformation
- Create and Draw Vertex/Index Buffer (Array in OpenGL)
- Restore World Transformation
- Disable Textures
- Disable Vertex Colors
- Disable Vertex Normals
- Disable Vertices
- Disable Lighting



รูปที่ 3.25 Renderer Class เพิ่มเติม

3.3.7 Billboard

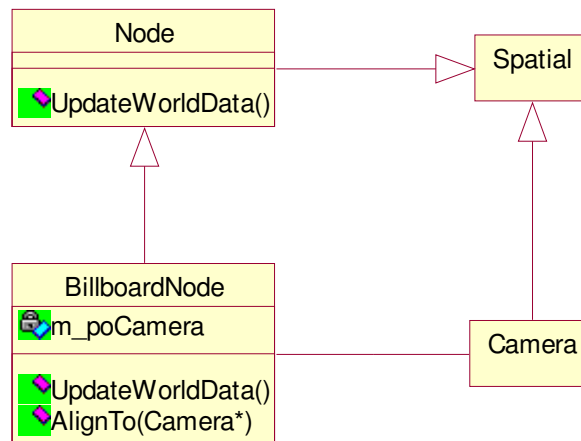
การทำ Billboard ไม่ได้มีอะไรมากไปกว่าการกำหนด World Transformation ของวัตถุให้มีทิศทางเข้าหากล้องซึ่งเดิมการทำ World Transformation เป็นหน้าที่ของ Scene graph (UpdateWorldData) อยู่แล้วการทำ Billboard ในระบบ Scene graph จึงไม่ใช่เรื่องยากเมื่อสามารถใช้ในการ Overridden method ที่ทำหน้าที่นั้นบน Node ได้ โดยสามารถคำนวณหามุมที่จะทำการหมุนวัตถุรอบแกน Y (Up) ของวัตถุนั้นได้ดังต่อไปนี้

ตำแหน่งของกล้องในเฟรมของ Billboard

$$\text{CamLoc} = \text{BillboardRotMat}^{-1} (\text{CameraWorldLocation} - \text{BillboardWorldLocation}) \quad (3.14)$$

ทิศของกล้องในแกน Y

$$\text{Angle} = \text{ATan}(\text{CamLoc.X}/\text{CamLoc.Z}) \quad (3.15)$$



รูปที่ 3.26 BillboardNode

3.3.8 Terrain

3.3.8.1 Large Terrain Rendering

เมื่อข้อมูลที่ใช้ใน Terrain ที่สวยงามใช้ได้จริงจะมีจำนวนมหาศาลวิธีการจัดการที่ดีที่สุดคือการแบ่ง Terrain ออกเป็นส่วนย่อยๆเรียกว่า Page ซึ่งตาม Scene Graph Hierarchy จะจัดไว้เป็นลูกของ Terrain (Node) และเป็น leaf node เสมอ (Geometry) ซึ่งใน Height Field Header ก็ต้องบอกด้วยว่ามีกี่ page (แถวและหลัก) และ page มีขนาดเท่าใดเพื่อความสะดวกต่อการคำนวณในภายหลัง (กำหนดให้เป็นสี่เหลี่ยมจัตุรัสขนาดเท่ากันหมด)

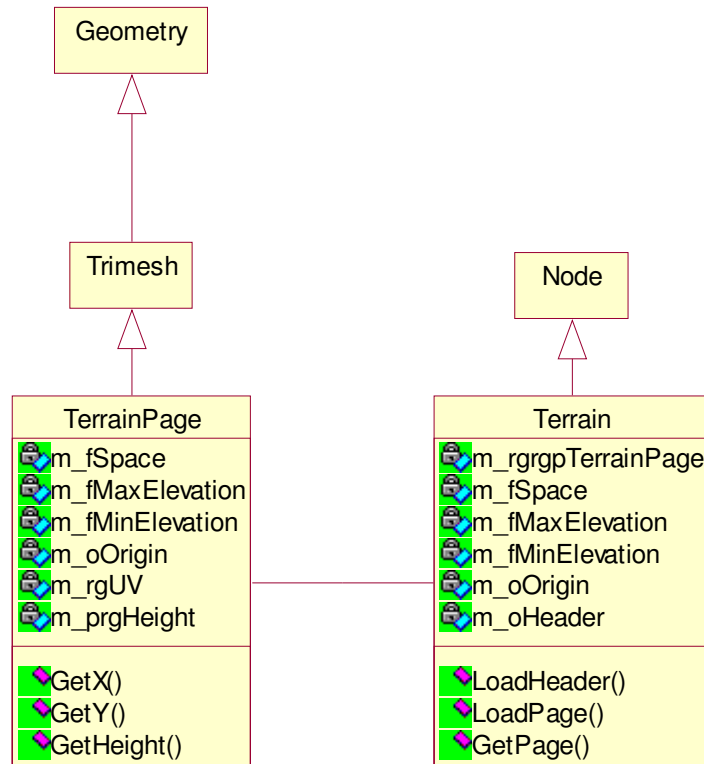
3.3.8.2 Height Field File Format

นอกจากจะให้ข้อมูลด้านความสูงของพื้นที่แล้ว ในส่วนหัวก็ต้องบรรจุ Header ลงไปด้วย เพื่อให้ Terrain Loader จะสามารถนำข้อมูลมาใช้อย่างถูกต้อง กำหนด Header ดังต่อไปนี้

```

struct TerrainHeader
{
    int iNumPages;           //จำนวน Page ทั้งหมด
    int iNumRows;            //จำนวนแถว
    int iNumCols;            //จำนวนหลัก
    int iPageSize;           //ขนาดของ Page
};
  
```

ถัดจาก Header จะเป็น Height Field ของแต่ละ Page เรียงจากซ้ายไปขวา และจากบนลงล่างเพื่อความสะดวกในการนำไปใช้ การสร้าง Terrain เริ่มจากอ่าน Header ว่ามีจำนวน Page เท่าใด จากนั้นสร้าง Page และติดตั้งเข้ากับ Terrain Node โดย Page ที่สร้างเป็น Plane ที่แบ่ง Segment ตามระยะห่างของแต่ละ Vertex ที่กำหนด โดยแต่ละ Vertex จะมีความสูงตามที่ปรากฏใน Height Field Data รวมถึงการคำนวณ UV ให้กับแต่ละ Vertex ด้วย

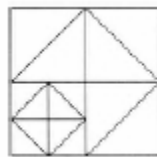


รูปที่ 3.27 Terrain

3.3.8.3 Continuous Level of Detail

ในการแสดงผล Terrain ที่มีความละเอียดสูงจะพบว่าในกรณีที่เป็นพื้นที่ราบไม่มีความจำเป็นต้องใช้ความละเอียดมากขนาดนั้น และบริเวณที่อยู่ใกล้ก็เช่นกัน ดังนั้นจึงมีวิธีที่จะลดความละเอียดของ Terrain ในกรณีที่เหมาะสมได้

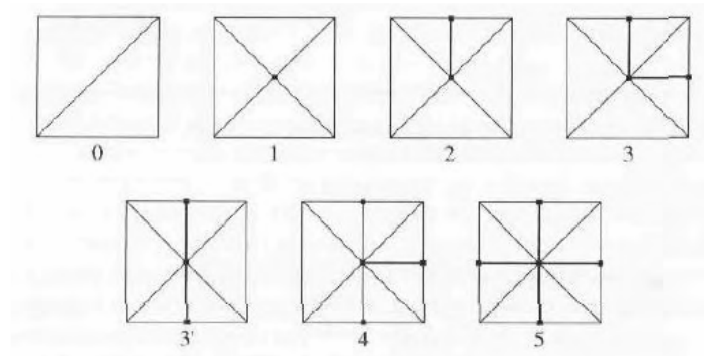
Page หนึ่ง Page จะถูกแบ่งเป็น Block ซึ่งขนาดขึ้นอยู่กับจำนวน pixel ที่ Block นั้นปรากฏบนจอภาพ Page หนึ่งๆ อาจจะถูกแบ่งเป็นหลาย Block ที่มีขนาดไม่เท่ากัน (Block Simplification) ดังภาพต่อไปนี้



รูปที่ 3.28 Block

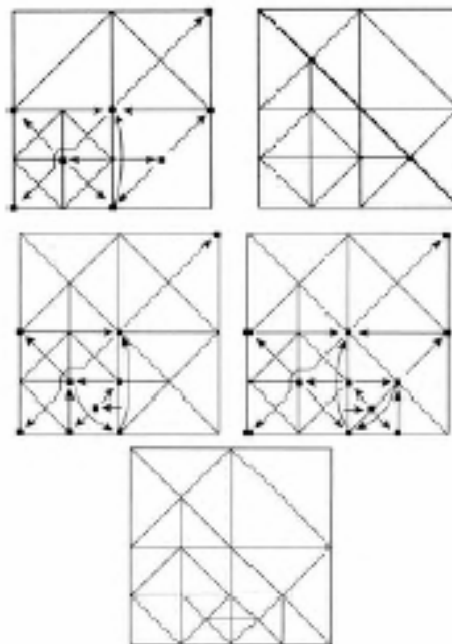
จากภาพด้านบนแสดง Page ซึ่งถูกแบ่งเป็น 7 Block ย่อย เมื่อต้องการจะแบ่ง Page ให้เป็น Block ได้อย่างลงตัว ซึ่ง Block มีขนาด 3×3 Page จึงถูกกำหนดให้มีขนาดเท่ากับ $2^n + 1$; $0 < n < 8$ ซึ่งขนาด Page ใหญ่ที่สุดคือ 129×129 สามารถแทนด้วย Block เพียงอันเดียวก็ได้ ซึ่งก็คือที่ตำแหน่ง 0, 64, 128 ทั้งแนวตั้งและแนวนอน การแบ่งจะทำทีละ 4 ซึ่งเห็นได้ชัดรูปแบบของ Quadtree โดย Block ที่กล่าวไปข้างต้นเป็น Root (Stride = 64) และแบ่งต่อมาเรื่อยๆ (Stride = Stride/2) จนถึงชั้น

ย่อยสุด คือใช้ทุก Vertex (Stride = 1) จากนั้นพิจารณาแต่ละ Block โดยใช้ Depth First Search เริ่มคำนวณขนาด Bounding Box ของแต่ละ Block ไปจนถึง leaf node ให้พิจารณาว่า Pixel Interval มีค่ามากถึงระดับหนึ่ง (Threshold) หรือไม่ถ้าถึงแสดงว่า ต้องการความละเอียดในระดับนี้ หากไม่ถึงให้พิจารณา Sibling node คว้าทุกตัวมีลักษณะเดียวกันหรือไม่ ถ้าใช่ให้ใช้ Parent Block นั้นเอง แต่ละ Block สามารถมีรูปแบบต่างๆกันขึ้นอยู่กับความเหมาะสมดังรูปต่อไปนี้



รูปที่ 3.29 Block Detail

จะเห็นได้ว่า Vertex บน Edge และ Center ของ Block สามารถ Enable หรือ Disable ได้ เพื่อให้เกิดเป็น Level of Detail Mesh ได้ดังรูป

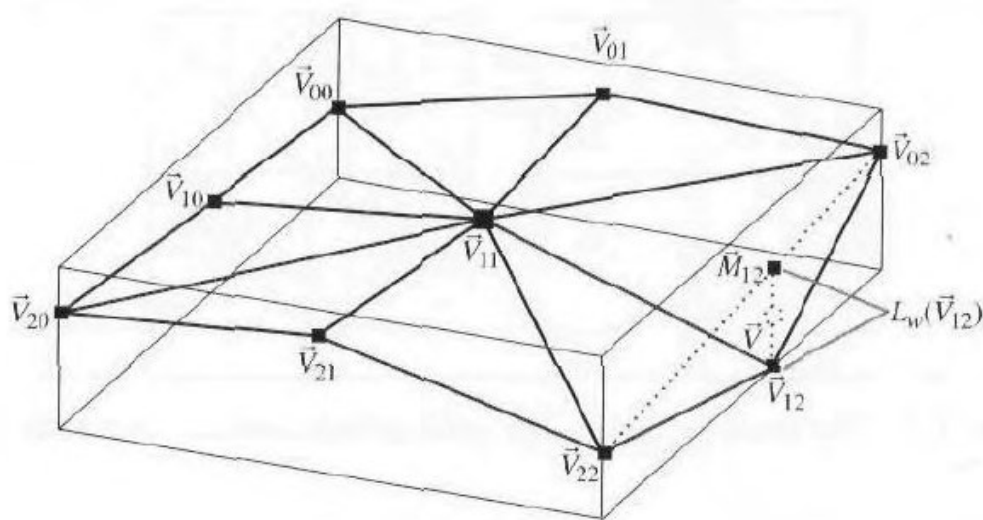


รูปที่ 3.30 Vertex Dependency

จากภาพด้านบนเห็นได้ว่าหาก Vertex หนึ่งหนึ่ง Enable จะต้องมี Vertex จำนวนหนึ่ง Enable ด้วย (Vertex Dependency) ซึ่งการทำเช่นนี้เรียกว่า Vertex Simplification

เมื่อแต่ละ Page มี Level of Detail ที่ไม่เท่ากันจึงต้องมีการปรับแก้โดยการสร้างและกำหนด Vertex Dependency เมื่อนำ Page มาเชื่อมต่อกัน (Page Stitching) อีกที เมื่อเชื่อมต่อกันได้

อย่างกลมกลืนแล้ว จะทำ Tessellation (Create Index) เป็นขั้นตอนสุดท้าย เพื่อส่งวัตถุเข้าสู่ Graphic pipeline ต่อไป

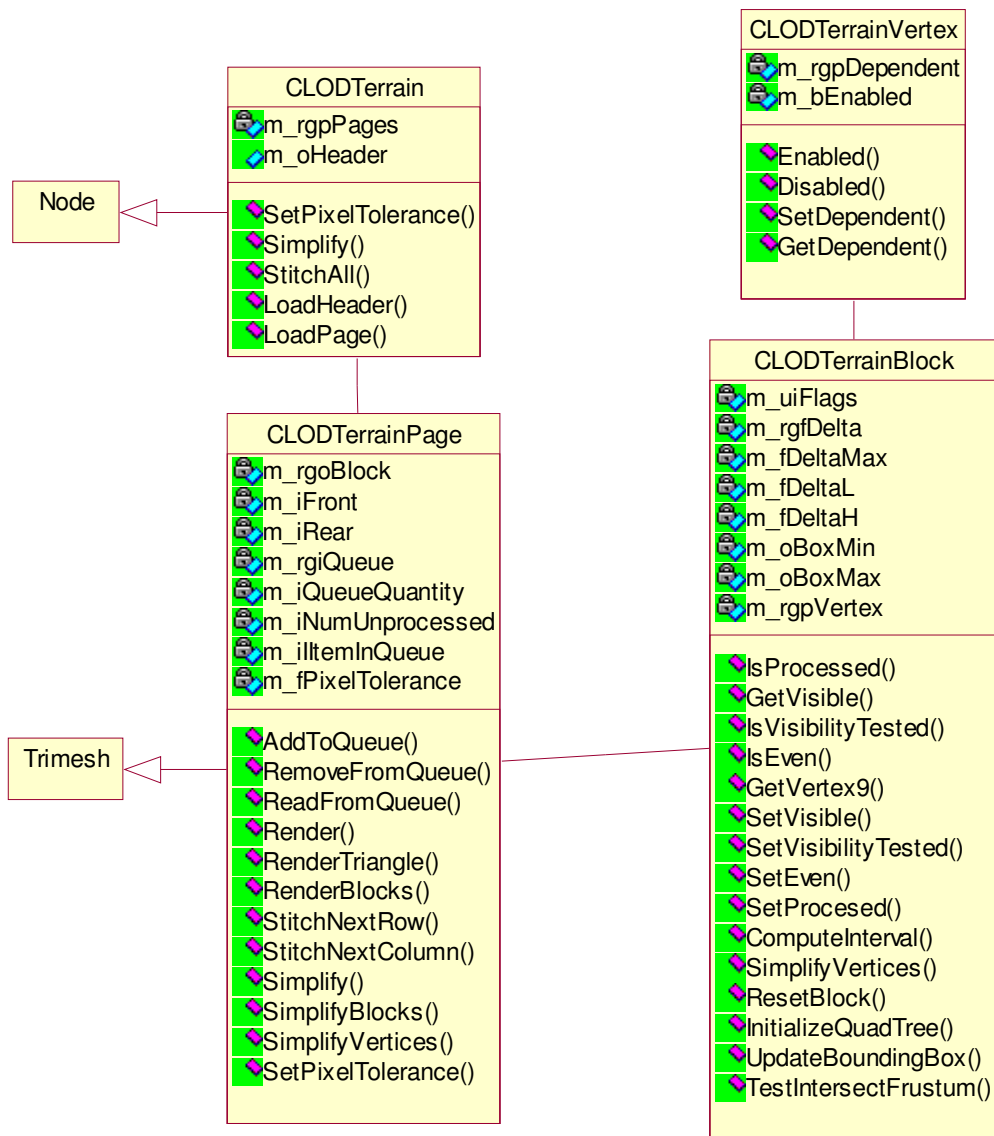


รูปที่ 3.31 Even Block

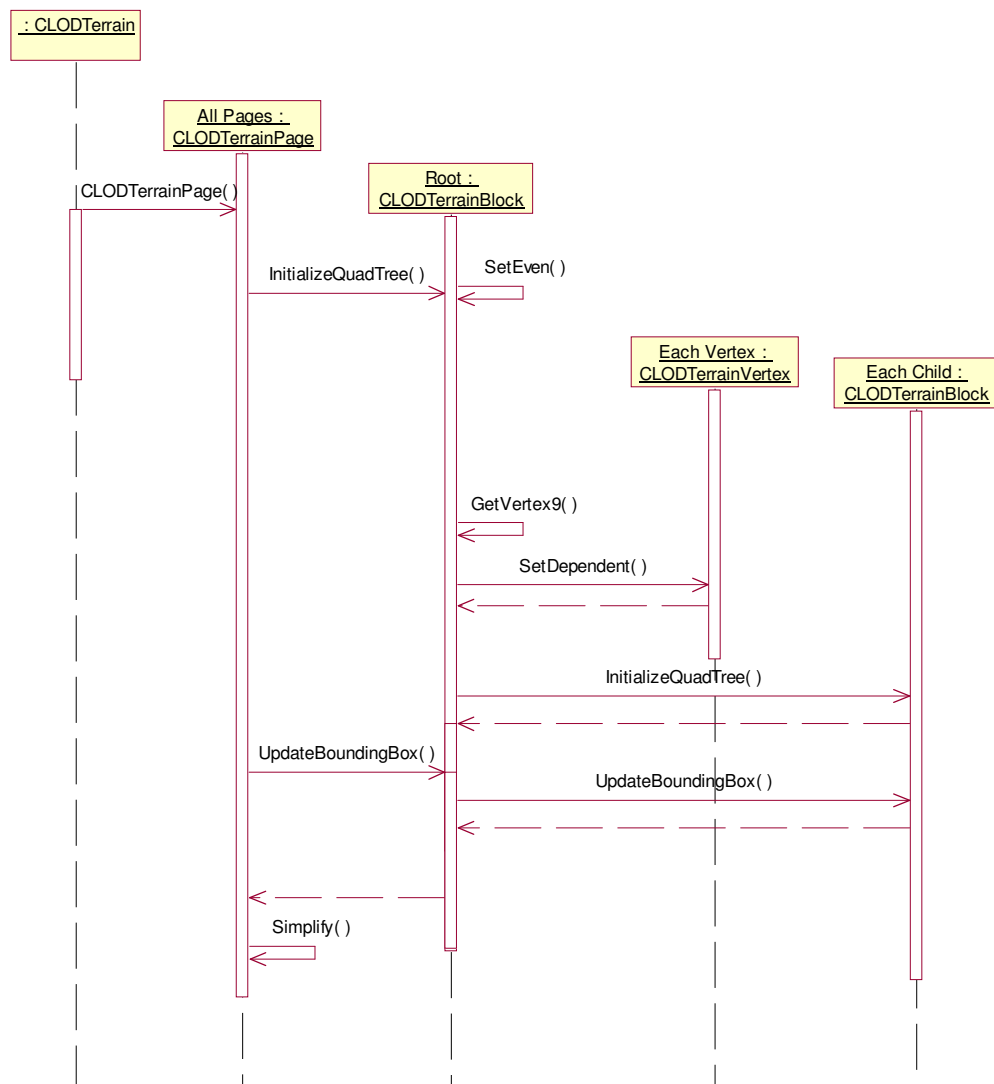
ในการทำ Block Based Simplification ภายหลังจากได้ Quad-tree ของ Block ทั้งหมด และใส่ Root Block เข้าไปใน Queue แล้วกระบวนการก็จะเริ่มจากการโปรเซส Block ที่เหลือใน Queue โดยตรวจสอบว่าเป็นลูกตัวแรกหรือไม่ ในกรณีของ Root Node ไม่ใช่ จึงตรวจสอบต่อว่าอินเตอร์เซกกับ View Frustum หรือไม่ หากใช่จะตรวจสอบต่อว่าขนาดในปัจจุบันยังสามารถแบ่งต่อได้อีกหรือไม่ ก็จะทำการคำนวณค่า Pixel Interval ของลูกแต่ละตัว หากมีตัวใดตัวหนึ่งที่มีค่า Interval สูงกว่าที่กำหนดก็จะทำการใส่ลูกทั้งหมดเข้าไปใน Queue และเริ่มตรวจใหม่เข้าในกรณีของลูกตัวแรกโดยหาก Node ในชั้นเดียวกันมีค่า Interval ต่ำกว่าที่กำหนดทั้งหมดก็จะทำการแทนค่าด้วย Parent ในกรณีนี้จะไม่เข้าเงื่อนไขจึงทำการ Sub divide ต่อไปอีกอย่างเช่นที่เกิดกับ Root และสิ้นสุดกระบวนการ (จำนวน Block ใน Queue ที่ยังไม่ได้ Process เป็น 0)

ในการทำ Vertex Based Simplification ภายหลังจากทำ Block Based Simplification แล้วจะมี Block ที่ใช้งานอยู่จริงใน Queue จะกระทำการต่อไปใน Block ที่มองเห็นได้ ดังในภาพที่ 3.31 เมื่อ V12 เป็น Candidate Vertex (เช่นเดียวกันกับ V01, V10 และ V21) ว่าจะสามารถ Disable ได้หรือไม่โดยการแปลงระยะ (M12, V12) เป็นระยะบน Screen Space (L_w) และเทียบ L_w ว่ามากกว่า Pixel Tolerance หรือไม่หากมากกว่าก็จะ Enable Vertex นั้นและส่งผลกับ Dependent Vertex ด้วย

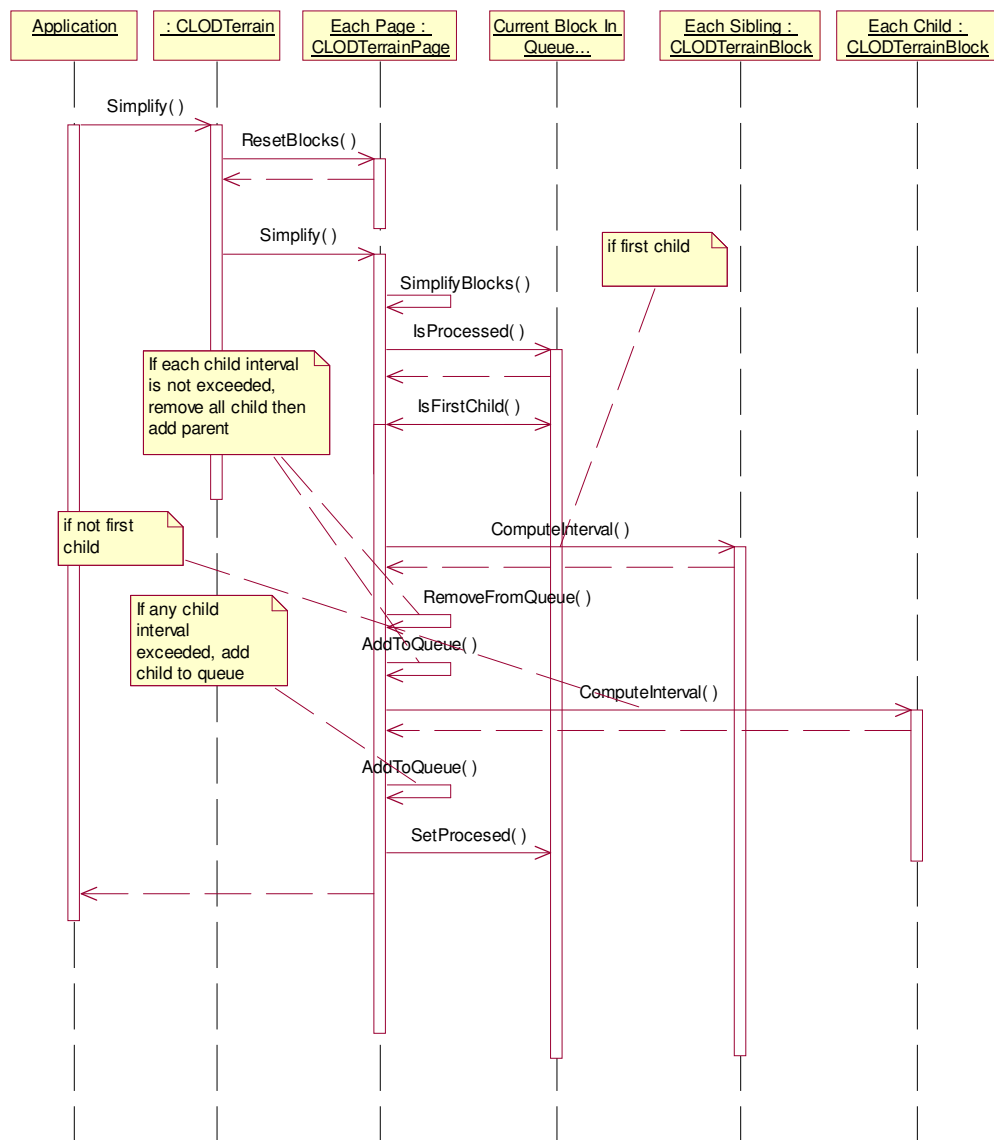
ส่วนในขั้นตอนของการ Tessellation นั้นจะทำการประมวล Block ทั้งหมดที่อยู่ใน Queue ในแต่ละ Block จะถูกวาดจาก 2 Triangle ในลักษณะ Recursive คือหาก Triangle ใดที่สามารถแบ่งต่อได้ และ Vertex ที่จะใช้แบ่งอยู่ในสถานะ Enabled ก็จะทำการเรียก RenderTriangle ต่อๆ ไปจนกระทั่งแบ่งต่อไม่ได้ แล้วจึงสร้าง Index ให้นักแสดงผล



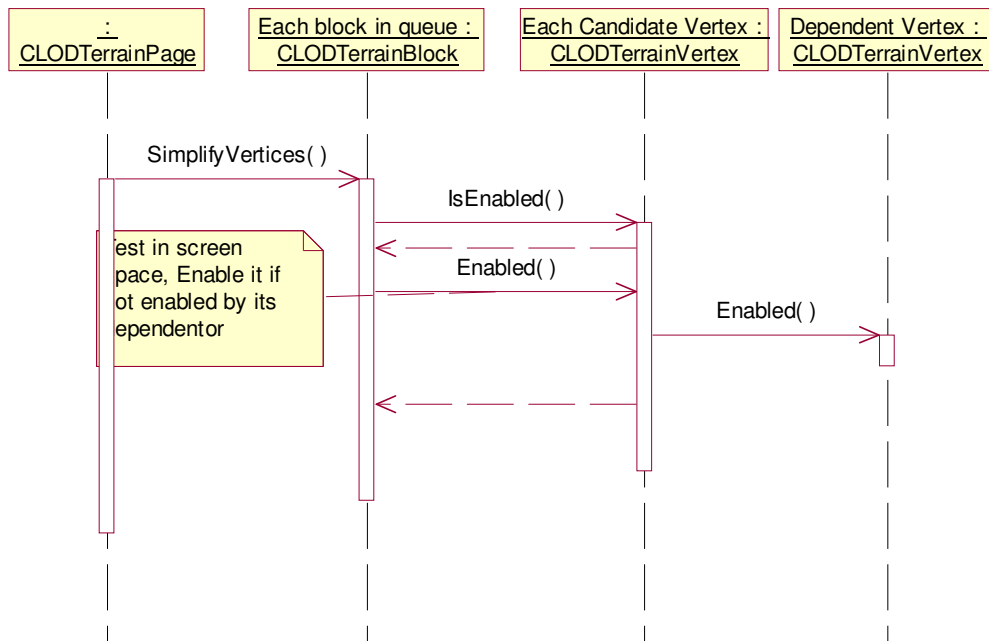
รูปที่ 3.32 CLOD Terrain



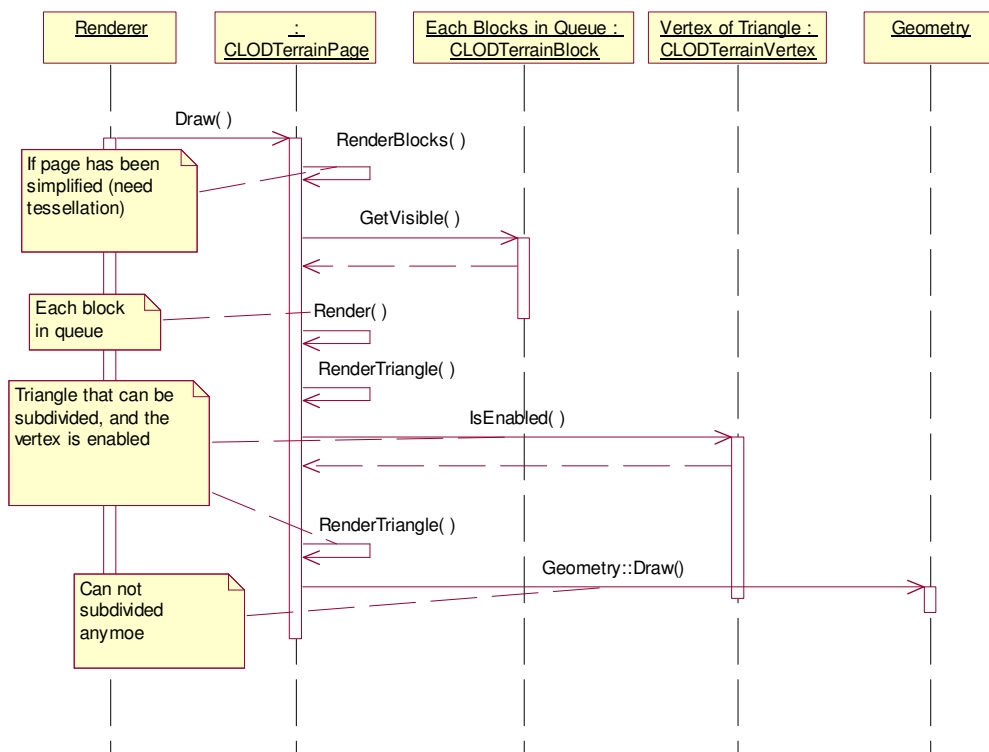
รูปที่ 3.33 Sequence diagram: Initialize



រូប 3.34 Sequence diagram: Block based simplification



รูปที่ 3.35 Sequence diagram: Vertex Simplification



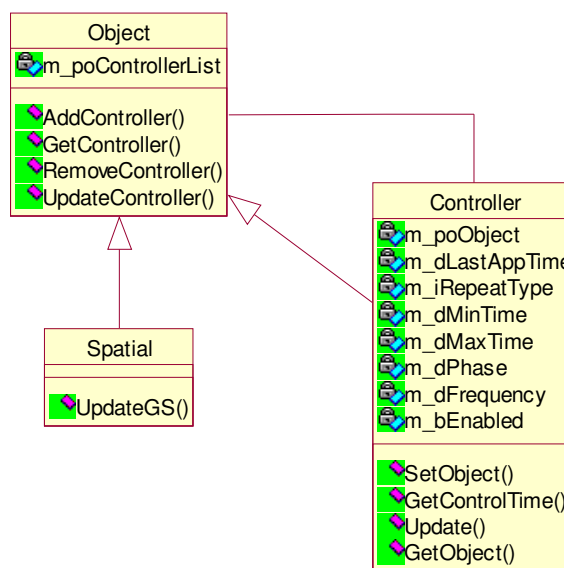
รูปที่ 3.36 Sequence diagram: Page Drawing

3.3.9 Controller

Animation หมายถึงการเปลี่ยนแปลงคุณสมบัติของวัตถุเมื่อเวลาเปลี่ยนแปลงไป ดังนั้นใน PC Engine จึงไม่ได้กำหนดให้การเปลี่ยนแปลงเหล่านี้สามารถเกิดขึ้นได้กับ Geometry เท่านั้น แต่ยังสามารถเกิดกับวัตถุชนิดต่างๆได้ โดยทำการกำหนดคลาส Controller ขึ้นมาควบคุมการเปลี่ยนแปลงเชิงเวลาที่เกิดขึ้นของวัตถุ โดยรับค่าเวลาผ่านทาง การ Update Geometry State

ของ Scene graph เพื่อที่จะมาปรับปรุงค่าต่างๆของ Object โดยการนำไปใช้ประโยชน์ที่เห็นได้ชัดเจนที่สุดคือการนำไปทำภาพเคลื่อนไหวนั่นเอง

ส่วนของคลาส Controller มีหน้าที่เพียงดูแล Controller Time เท่านั้น โดยจะมีวิธีการวนรอบแบบต่างๆเช่น การวนกลับมาจุดแรกใหม่(Repeat) การย้อนกลับมาจนถึงจุดแรกเมื่อถึงจุดจบ(Cycle) หรือการจบเลย(Clamp) อย่างไรก็ตามในส่วนของเมธอด UpdateWorldData หรือ UpdateGS ในคลาส Spatial ก็ต้องทำการเรียก UpdateController ด้วยเพื่อนำค่าเวลาในขณะนั้นไปแปลงเป็นค่าเวลาเฉพาะของ Controller

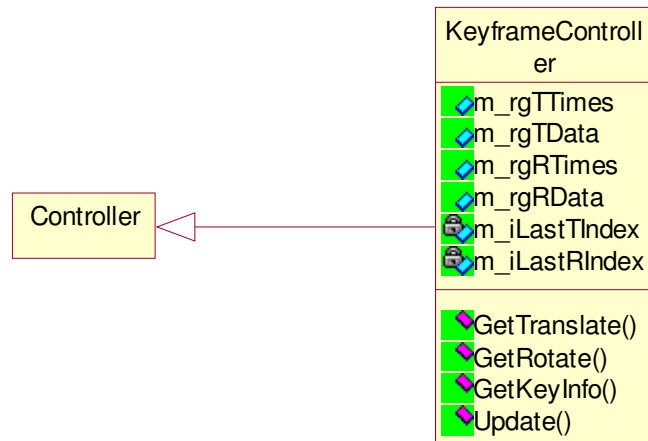


รูปที่ 3.37 Controller Class

3.3.10 Keyframe Animation

คือการกำหนดการเคลื่อนไหวของวัตถุในแต่ละ Shot ที่สำคัญออกมา (คล้ายกับการวาดการ์ตูนเคลื่อนไหว) โดยสามารถกำหนด Local Transformation ของวัตถุในแต่ละ frame ได้ โดยกำหนดเวลาที่จะใช้ของแต่ละ frame ไปด้วย หากเวลาที่ระบุโดย Application มีค่าระหว่างช่วงเวลาที่กำหนด ก็จะทำการ Linear Interpolate ค่า Transformation ระหว่างช่วงเวลานั้นและทำการ Apply ให้กับวัตถุ

Transformation ที่สนับสนุนใน Keyframe ได้แก่ Translation ซึ่งอยู่ในรูปของ Vector และ Rotation ซึ่งอยู่ในรูปของ Quaternion



รูปที่ 3.38 Keyframe Controller

3.3.11 Skin Controller

จากข้อเสียของ Keyframe Animation ก็คือต้องเก็บ Transformation ของแต่ละ Vertices ในวัตถุ หากนำไปใช้ในโมเดลรูปคนซึ่งมีจำนวน Vertices มากๆ ก็จะต้องเปลืองทรัพยากรในการเก็บ และการ Apply Transformation มาก จึงได้นำแนวคิดของ Bone มาใช้โดยจะทำ Keyframe animation เฉพาะส่วนที่เป็น Bone ของวัตถุเท่านั้น และใช้ความสัมพันธ์ระหว่าง Vertex กับ Bone เป็นตัวกำหนด Transformation ของแต่ละ vertex อีกทีความสัมพันธ์มีดังต่อไปนี้

- B : Bone(s) ที่มีผลต่อ Vertex
- W : Weight ของ Bone(s) ที่มีผลต่อ Vertex
- O : Offset ของ Vertex จาก Bone(s)

ยกตัวอย่างตารางความสัมพันธ์

Vertex	Bone0	Bone1	Bone2	Bone3
V0	W_{00}, O_{00}	-	W_{02}, O_{02}	W_{03}, O_{03}
V1	-	W_{11}, O_{11}	-	W_{13}, O_{13}
V2	W_{20}, O_{20}	-	-	W_{23}, O_{23}

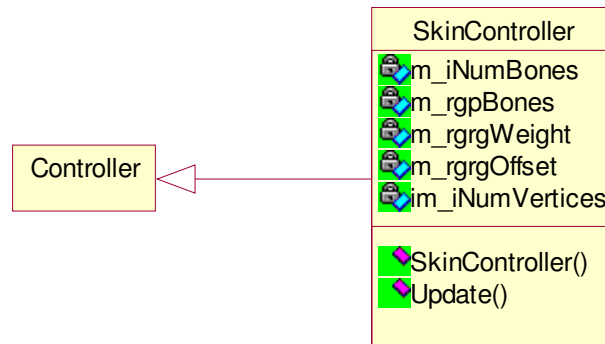
ตารางที่ 3.5 ตัวอย่าง Bone Offset และ Weight

เมื่อทำการ Update ความสัมพันธ์จะได้สมการดังต่อไปนี้

$$\begin{aligned}
 V0 &= W_{00} (S_0 R_0 O_{00} + T_0) + W_{02} (S_2 R_2 O_{02} + T_2) + W_{03} (S_3 R_3 O_{03} + T_3) \\
 V1 &= W_{11} (S_1 R_1 O_{11} + T_1) + W_{13} (S_3 R_3 O_{13} + T_3) \\
 V2 &= W_{20} (S_2 R_2 O_{20} + T_2) + W_{23} (S_3 R_3 O_{23} + T_3)
 \end{aligned} \tag{3.16}$$

Skin Controller จึงไม่ได้ยุ่งเกี่ยวกับทางด้านเวลาแต่จะทำการปรับ Vertex Data ให้เป็นตำแหน่งที่กำหนดจาก Offset และ Weight จึงต้องทำการแก้ไข World Transformation ให้เป็น

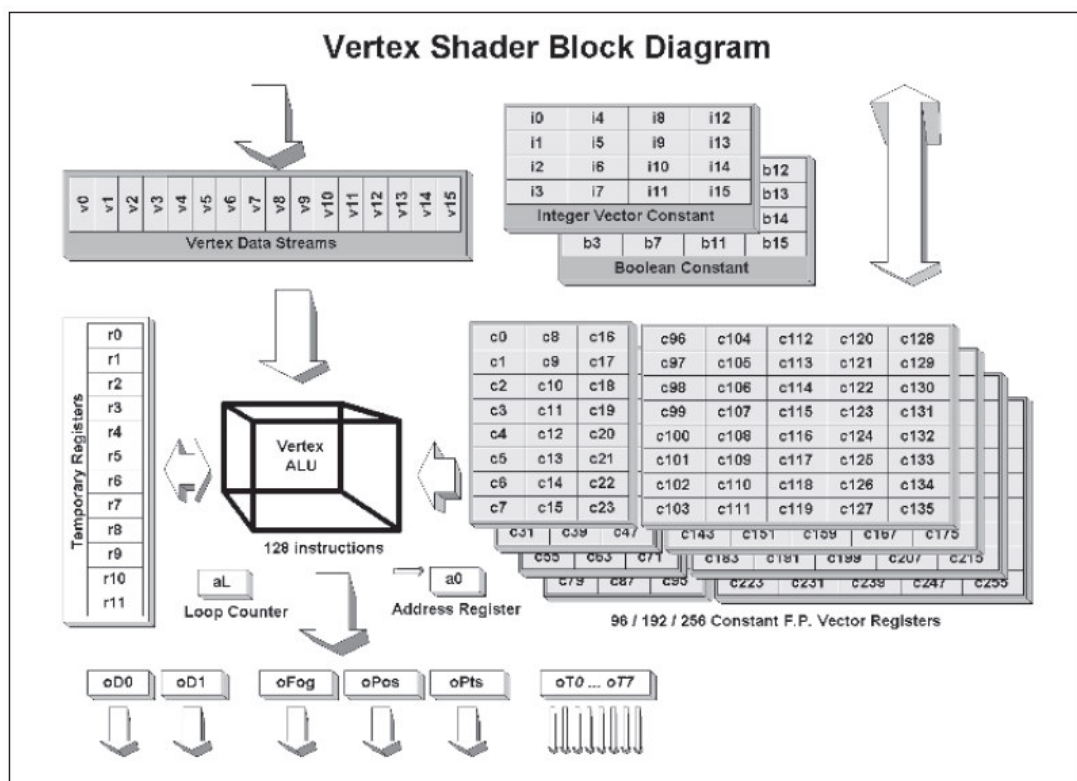
Identity และกำหนดไม่ให้เปลี่ยนแปลง ซึ่งต้องปรับปรุง Class Spatial เล็กน้อยเพื่อสามารถบอกได้ว่าไม่ต้องทำการ Apply World Transformation ให้

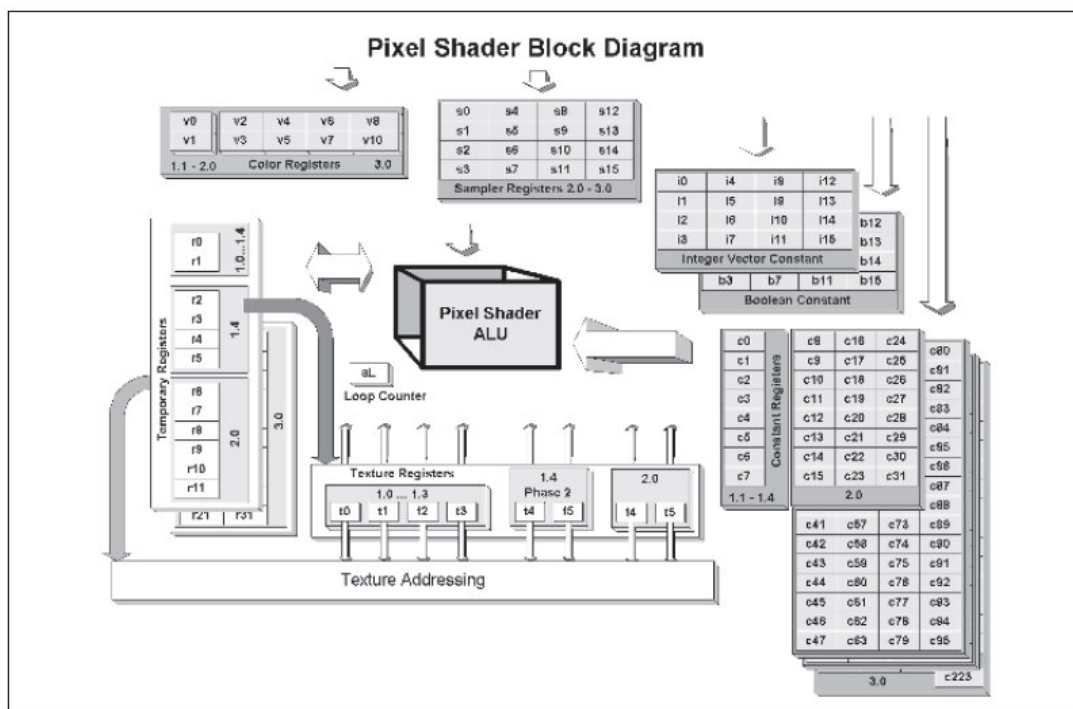


รูปที่ 3.39 Skin Controller

3.3.12 Shader

ใน 3D Pipeline ช่วงของกระบวนการ Transform & Lighting และ Texturing นั้นจะเปิดโอกาสให้ผู้ใช้งานสามารถทำการ Program เองได้ด้วย Vertex/Pixel Shader เนื่องจากการใช้ Vertex หรือ Pixel Shader ไม่สามารถใช้ Drawing Function เดิม (`DrawPrimitive`) ได้เนื่องจากต้องติดตั้ง Shader Program ให้กับ Derived Renderer จึงจัดให้ Shader เป็น Effect ชนิดหนึ่ง และเพิ่ม `DrawShader` เป็น Drawing Function ขึ้นใน Renderer โดยเพิ่มส่วนของการติดตั้ง Shader Program และใน ShaderEffect ก็จะประกอบไปด้วย VertexShader และ PixelShader ซึ่งสามารถทำ Generalize ได้เป็น Class Shader ที่จะประกอบไปด้วยส่วน Code ของโปรแกรม และลิสต์ของ Shader Constants ต่างๆ





รูปที่ 3.40 Vertex/Pixel Shader Block Diagram

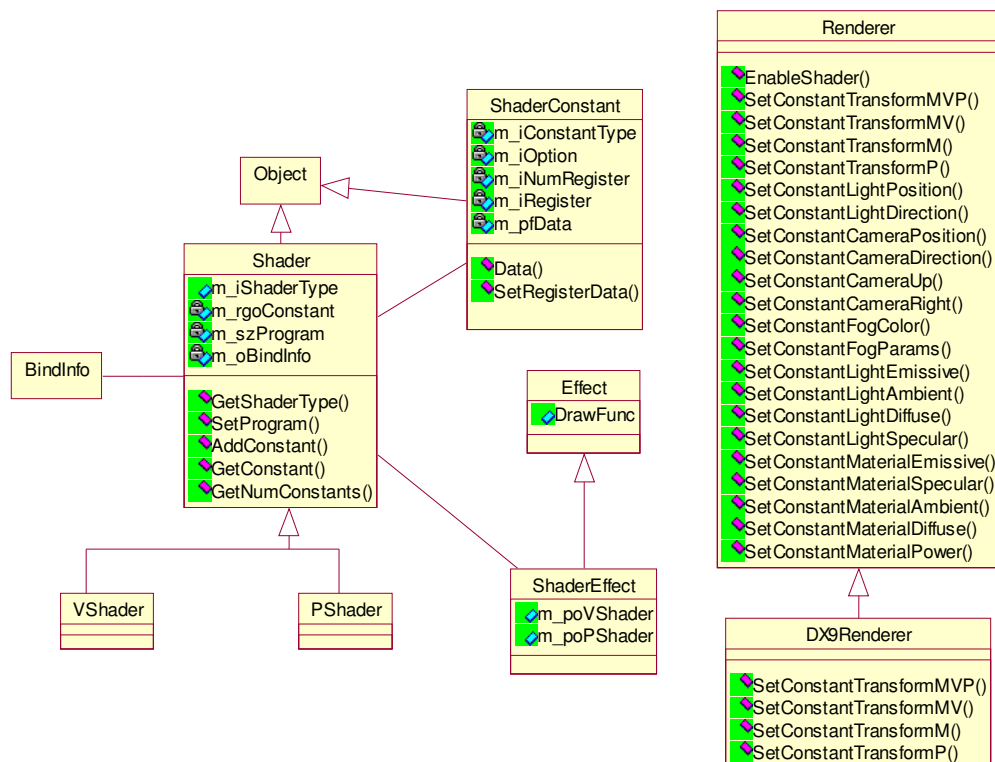
ส่วนของ Shader Constants ในการใช้ Vertex และ Pixel Shader จะต้องมีการกำหนดค่าให้ Constant Register (c) เพื่อนำไปใช้ในโปรแกรมอย่างเหมาะสม เพื่อความสะดวกของผู้ใช้งานค่าโดยทั่วไปที่จะทำการตั้งให้กับ Constant Register เช่น World-View-Projection Matrix ก็เป็นค่าที่มีการเปลี่ยนแปลงอยู่ตลอดเวลา และเป็นสิ่งที่ Renderer รู้อยู่แล้วจึงกำหนดให้ Renderer ตรวจประเภทของ Constant ที่ติดตั้งไว้และทำการกำหนดค่าให้ตามความเหมาะสมดังตารางที่ 3.6 โดยในกรณีที่เป็น Matrix จะต้องสามารถเลือกได้ว่าจะให้อยู่ในรูปแบบใดคือ Inverse Transpose หรือ Inverse และ Transpose ส่วนในกรณีเกี่ยวกับแสงก็จะต้องระบุลำดับที่ด้วย

ประเภทของ Constant	คำอธิบาย
TRANSFORM_M	Model Transformation
TRANSFORM_P	Projection Transformation
TRANSFORM_MV	Model-View Transformation
TRANSFORM_MVP	Model-View-Projection Transformation
CAMERA_POSITION	ตำแหน่งกล้อง
CAMERA_DIRECTION	ทิศทางการกล้อง
CAMERA_UP	ทิศทางด้านบนของกล้อง
CAMERA_RIGHT	ทิศทางด้านขวาของกล้อง
FOG_COLOR	สีของหมอก
FOG_PARAMS	(start, end, density, enabled)
MATERIAL_EMISSIVE	Material Emissive

MATERIAL_AMBIENT	Material Ambient
MATERIAL_DIFFUSE	Material Diffuse
MATERIAL_SPECULAR	Material Specular
MATERIAL_SHININESS	Material Shininess
LIGHT_POSITION	ตำแหน่งของแสง
LIGHT_DIRECTION	ทิศทางของแสง
LIGHT_AMBIENT	แสง Ambient
LIGHT_DIFFUSE	แสง Diffuse
LIGHT_SPECULAR	แสง Specular
LIGHT_ATTENPARAMS	ค่า Attenuate ของแสง
NUMERICAL_CONSTANT	ค่าคงที่ตัวเลข
USER_DEFINED	ค่าที่ User กำหนดเอง

ตารางที่ 3.6 Shader Constant มาตรฐาน

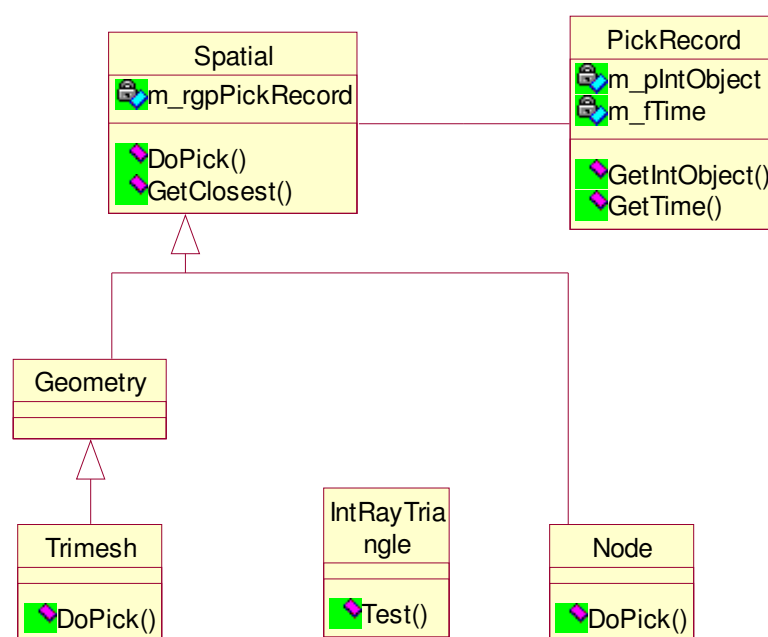
เช่นเดียวกันกับ Texture เพื่อเพิ่มประสิทธิภาพในการจัดการหน่วยความจำ Shader Program ใดที่ถูกคอมไพล์และติดตั้งแล้วก็ไม่จำเป็นต้องคอมไพล์และติดตั้งอีก จึงเพิ่ม BindInfo ให้กับคลาส Shader



รูปที่ 3.41 Shader Support

3.3.13 Detailed Picking

ภายหลังจากการสร้าง Picking Ray จากคลาส Camera แล้ว ผู้ใช้สามารถที่จะทำ Picking แบบหยาบได้โดยตรวจสอบกับ Bounding Volume ของแต่ละวัตถุโดยตรง แต่จากคุณลักษณะของ Scene Graph สามารถทำ Picking แบบลำดับขั้นได้ โดยใช้คุณสมบัติ Virtual Function ที่จะเลือกการกระทำได้ ได้แก่ ถ้าเป็น Node ก็จะถ่ายทอดคำสั่งต่อไปให้ลูก และถ้าเป็น Trimesh ก็จะต้องทำการตรวจกับ Bounding Volume ก่อนแล้วจึงตรวจสอบในระดับ Triangle ผลของการทำ Picking คือ Pick Record Array ซึ่งประกอบไปด้วยวัตถุที่ตัด และค่าของเวลาที่ Ray ตัด



รูปที่ 3.42 Picking Classes

3.4 Physically Base Simulation Module

หลักการออกแบบส่วนของ Physically Base Simulation Module ประกอบไปด้วยส่วนหลักสามส่วนด้วยกันคือ

1. ส่วนตรวจหาการชนกันของวัตถุ(Collision Detection)
2. ส่วนคำนวณการเคลื่อนที่ของวัตถุ (Rigidbody System)
3. ส่วนหา ปฏิสัมพันธ์ของวัตถุหลังการชนกันของวัตถุ(Collision Response)

โดยทั้งสามส่วนจำเป็นที่จะต้องพึ่งพาอาศัยกัน ในทุกครั้งที่มีการเคลื่อนที่จะมีการตรวจสอบการชนกันของวัตถุ และเมื่อชนกันก็จะส่งวัตถุที่ชนกันนั้นไป ส่วน ปฏิสัมพันธ์ เพื่อคำนวณหาทิศทางใหม่ของวัตถุที่ชนกันต่อไป โดยในส่วน คำนวณการเคลื่อนที่กับส่วนหาปฏิสัมพันธ์หลังการชน เนื่องจากว่า ส่วนปฏิสัมพันธ์นั้นจะรับค่าเป็นวัตถุแข็งแกร่ง ที่ชนกันการตรวจสอบ การชนนั่นเอง

3.4.1 ส่วนตรวจหาการชนกันของวัตถุ (Collision Detection)

ใช้หลักการในการหาการชนกันของวัตถุสองชิ้น โดยตรวจสอบจากข้อมูลของวัตถุทั้งสองซึ่งองค์ประกอบหลักของส่วนนี้จะถูกออกแบบให้เป็น คลาส ในการออกแบบคลาสนี้ ได้คำนึงถึงวัตถุที่จะมาหาว่าชนกันนั้นมีทั้งแบบที่เคลื่อนที่และแบบที่หยุดนิ่ง ดังนั้นเราจึงจำแนกการตรวจจับการชนออกมาเป็น การตรวจจับการชนของวัตถุที่เคลื่อนที่ (มีความเร็ว) และการตรวจจับการชนของวัตถุที่หยุดนิ่ง (ไม่มีความเร็ว) และเพื่อประสิทธิภาพเอนจินนี้ได้ทำการแยกการทดสอบและการหาตำแหน่งที่ชน ออกจากกันเพื่อบางที่อาจต้องการรู้ว่าชนกับไม่ชนจะได้ไม่ต้องไปเข้าสมการคณิตศาสตร์เพื่อหาจุดของตำแหน่งที่ชน ดังนั้นคลาสทุกคลาสก็จะมีฟังก์ชันที่เหมือนกันและตัวแปรบางตัวแปรที่เหมือนกัน เราจึงได้สร้างคลาสต้นแบบขึ้นเพื่อให้คลาสต่างๆมา derive จากคลาสนี้

```

Class Intersector
{
public:
    virtual ~Intersector();

    //static intersection queries
    virtual bool Test();
    virtual bool Find();

    //dynamic intersection queries
    virtual bool Test(const float fTMax,const Vector3& vVelocity0,const Vector3& vVelocity1);
    virtual bool Find(const float fTMax,const Vector3& vVelocity0,const Vector3& vVelocity1);

    //time at which two object are in first constant for dynamic intersection
    float GetContactTime() const;

protected:
    Intersector();
    float m_fContactTime;
};

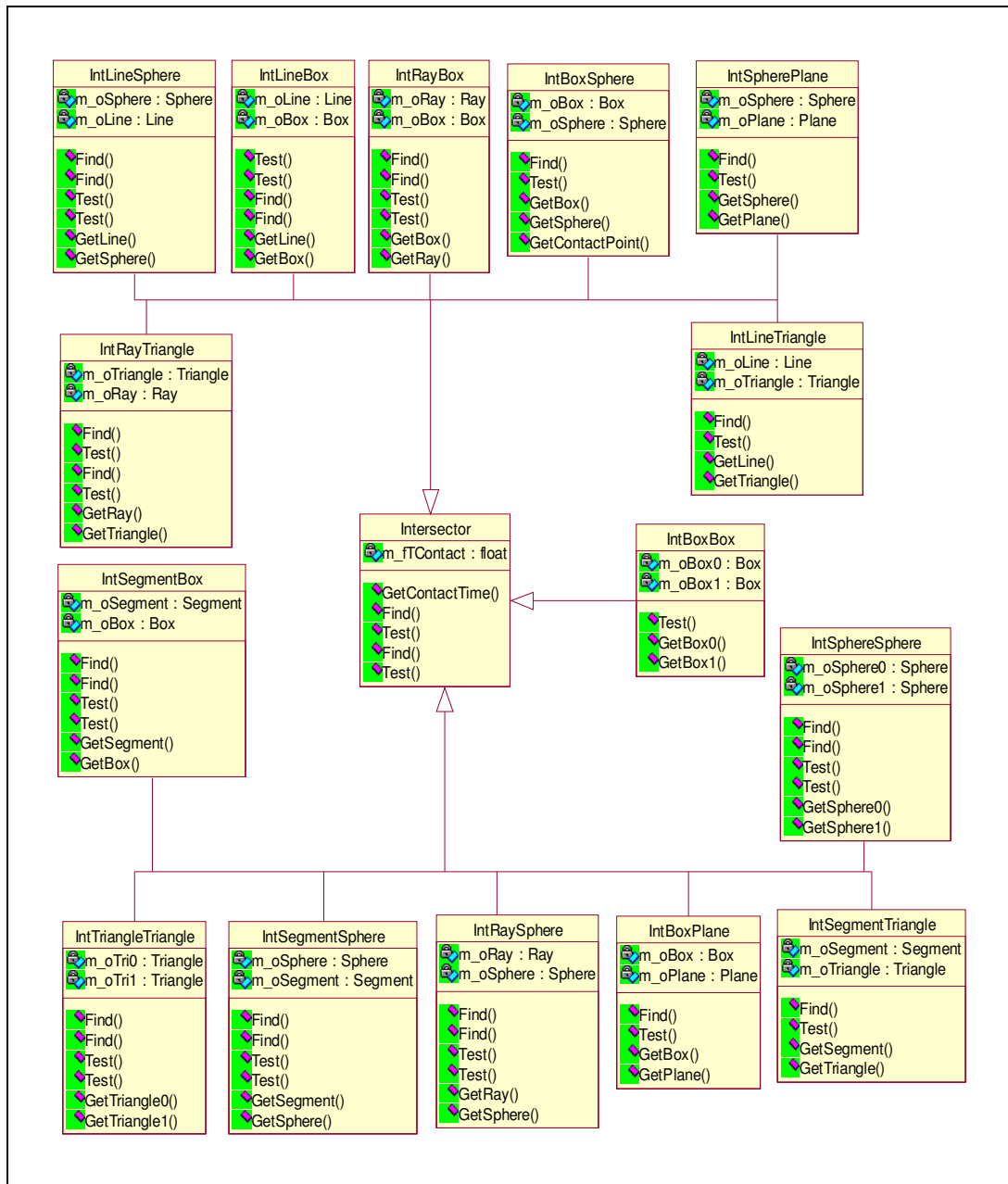
```

จากโค้ด จะเห็นได้ว่า ได้ออกแบบให้ มีฟังก์ชัน Test() และ Find () สองลักษณะด้วยกันคือ แบบไม่มีการเคลื่อนที่ของวัตถุกับแบบที่มีการเคลื่อนที่ของวัตถุโดยถ้าเป็นแบบ

ที่มีการเคลื่อนที่ของวัตถุจะให้ค่าของความเร็วของวัตถุทั้งสองและเวลาในส่วนของ ฟังก์ชัน Test() เป็นการตรวจว่ามีการชนกันหรือไม่ ถ้ามีการชนจะคืนค่า true ถ้าไม่มีการชนจะคืน false โดยไม่มีการเก็บค่าใดๆเกี่ยวกับการชนที่เกิดขึ้น และ ในส่วนของ ฟังก์ชัน Find() ถ้ามีการชนจะมีการเก็บเวลาที่ชน และค่าต่างๆ ที่เกี่ยวกับการชนโดยค่าต่าง ๆ นั้นขึ้นอยู่กับว่าเป็นการชนระหว่าง วัตถุประเภทใด โดยคลาสที่ Derive มาจากคลาส Intersector ประกอบไปด้วย

1. การตรวจสอบการชนระหว่างเส้นตรงกับสามเหลี่ยม
2. การตรวจสอบการชนระหว่างเรย์ กับสามเหลี่ยม
3. การตรวจสอบการชนระหว่างส่วนของเส้นตรงกับสามเหลี่ยม
4. การตรวจสอบการชนระหว่างเส้นตรงกับทรงกลม
5. การตรวจสอบการชนระหว่างเรย์ กับทรงกลม
6. การตรวจสอบการชนระหว่างส่วนของเส้นตรงกับทรงกลม
7. การตรวจสอบการชนระหว่างเส้นตรงกับสี่เหลี่ยม
8. การตรวจสอบการชนระหว่างเรย์ กับ สี่เหลี่ยม
9. การตรวจสอบการชนระหว่างส่วนของเส้นตรงกับสี่เหลี่ยม
10. การตรวจสอบการชนระหว่างสามเหลี่ยมกับสามเหลี่ยม
11. การตรวจสอบการชนระหว่างทรงกลมกับทรงกลม
12. การตรวจสอบการชนระหว่างสี่เหลี่ยมกับสี่เหลี่ยม
13. การตรวจสอบการชนระหว่างสี่เหลี่ยมกับทรงกลม
14. การตรวจสอบการชนระหว่างสี่เหลี่ยมกับระนาบ
15. การตรวจสอบการชนระหว่างทรงกลมกับระนาบ

รูปที่ 3.43 เป็นตัวอย่างการออกแบบคลาสที่ใช้ในการตรวจสอบการชนที่ Derive มาจาก คลาส Intersector()



รูปที่ 3.43 แสดงคลาสที่ Derive มาจาก คลาส Intersector

ในการตรวจสอบวัตถุที่ชนกันนั้นสามารถทำได้ละเอียดสุดในระดับ สามเหลี่ยมกับสามเหลี่ยม โดยอาศัยการสร้าง Bounding Volume Tree มาช่วยในการเพิ่มประสิทธิภาพในการตรวจสอบ ซึ่งจะมีการเก็บสามเหลี่ยมที่เป็นองค์ประกอบของวัตถุให้อยู่ในรูปของ Tree เพื่อลดการวนรอบในการตรวจสอบระหว่างสามเหลี่ยมกับสามเหลี่ยม โดยมีคลาสที่ทำหน้าที่รับผิดชอบคือ Record ซึ่ง Record นี้จะเป็นคลาสที่ทำการเก็บ สามเหลี่ยมทั้งหมดของวัตถุนั้นๆ องค์ประกอบของคลาสดังนี้

```

Class CollisionRecord
{
Public:
    typedef void (*Callback) (CollisionRecord& roRecord0, int iT0,
CollisionRecord& roRecord1, int iT1, Intersector* pIntersector);

    CollisionRecord(Trimesh* pTrimesh, BoundingBox* pTree,
        Vector3* pvVelocity, Callback oCallback, void* poCallbackData);

    ~CollisionRecord();

    Trimesh* GetMesh();
    Vector3* GetVelocity();
    void* GetCallbackData();

    void TestIntersection (CollisionRecord& roRecord);
    void FindIntersection (CollisionRecord& roRecord);
    void TestIntersection (float fTMax, CollisionRecord& roRecord);
    void FindIntersection (float fTMax ,CollisionRecord& roRecord);
protected:
    Trimesh* m_pTrimesh;
    BoundingBox* m_pTree;
    Vector3* m_pvVelocity;
    Callback m_oCallback;
    void* m_pvCallbackData;
}

```

หลักการตรวจสอบก็คือ จะทำการตรวจสอบสามเหลี่ยมทุกสามเหลี่ยมระหว่างสองวัตถุและเมื่อพบว่าชนกันก็จะเรียก CallBack ฟังก์ชัน โดยเราสามารถกำหนดฟังก์ชันนี้ได้ว่าต้องการให้ทำอะไรกับ วัตถุสามเหลี่ยมนั้น เนื่องจากการตรวจสอบแบบนี้จะใช้ความสามารถของเครื่องสูงเราจึงมีการ นำเทคนิคมาใช้ในการตรวจสอบการชนคือการแบ่งกลุ่มการตรวจสอบการชนกับระหว่างวัตถุ โดยอาศัยหลักการที่ว่า ถ้าวัตถุที่ไม่สามารถจะเคลื่อนที่มาชนกันได้เราก็จะไม่ทำการตรวจสอบการชน โดยการจัดกลุ่มของวัตถุที่สามารถเคลื่อนที่มาชนกันได้เป็นกลุ่มๆ แล้วทำการตรวจสอบการชนกันเฉพาะภายในกลุ่มเท่านั้น เป็นการช่วยเพิ่มประสิทธิภาพในการตรวจสอบการชนซึ่งเราได้ทำการออกแบบคลาส กลุ่มของวัตถุที่มีความเป็นไปได้ที่จะชนกันขึ้น โดยรูปแบบของคลาสเป็นดังนี้

```

Class CollisionGroup
{
public:
    CollisionGroup();
    ~CollisionGroup();

    bool Add(CollisionRecord* pRecord);
    bool Remove(CollisionRecord* pRecord);

    void TestIntersection();
    void FindIntersection();

    void TestIntersection (float fTMax);
    void FindIntersection (float fTMax);

protected:
    Array<CollisionRecord*> m_prgRecord;
}

```

ภายในฟังก์ชัน TestIntersect นั้นจะทำการตรวจสอบระหว่างวัตถุทั้งหมดที่อยู่ในกลุ่มนี้ โดยการเพิ่มวัตถุ จะทำผ่านฟังก์ชัน Add และการเอาออกก็จะผ่านฟังก์ชัน Remove ซึ่งวัตถุที่นำเข้าไปนั้นต้องเป็น Collision Record เพราะว่าเป็นคลาสที่สร้างขึ้นมาเพื่อสนับสนุน Collision Record นั้นเอง

3.4.2. ส่วนคำนวณการเคลื่อนที่ของวัตถุ (Rigid Body Simulation)

ในส่วนนี้จะเป็นการสร้างวัตถุแข็ง โดยจะจำลองโดยมีเก็บค่าของ ตำแหน่ง ความเร็วเชิงเส้น ความเร็วเชิงมุม โมเมนตัมเชิงเส้น โมเมนตัมเชิงมุม น้ำหนัก ค่าความเฉื่อยของวัตถุ แรงธรรมชาติ ทั้งในเชิงเส้นและเชิงมุม แล้วนำค่าเหล่านี้มาคำนวณตำแหน่งของวัตถุ ณ เวลาถัดไป รวมไปถึง ลักษณะการเคลื่อนที่ของวัตถุด้วย ซึ่งคลาสนี้จะมีรูปแบบดังนี้

```

Class RigidBody
{
public:
    RigidBody();
    virtual ~RigidBody();

    Vector3& Position();

    void ComputeMassProperties (const Vector3* rgvVertex,int iTQuantity,const unsigned int*
rgiIndex,Vector3& rvCenter);

    //set rigid body state

```



```

void SetMass (float fMass);
void SetBodyInertia (const Matrix3& roInertia);
void SetPosition (const Vector3& roPos);
void SetQOrientation (const Quaternion& roQOrient);
void SetLinearMomentum (const Vector3& roLinMom);
void SetAngularMomentum (const Vector3& roAngMom);
void SetROrientation (const Matrix3& roROrient);
void SetLinearVelocity (const Vector3& roLinVel);
void SetAngularVelocity (const Vector3& roAngVel);

```

```

//get rigid body state
float GetMass () const;
float GetInverseMass () const;
const Matrix3& GetBodyInertia () const;
const Matrix3& GetBodyInverseInertia ()const;
Matrix3 GetWorldInertia ()const;
Matrix3 GetWorldInverseInertia ()const;
const Vector3& GetPosition() const;
const Quaternion& GetQOrientation() const;
const Vector3& GetLinearMomentum() const;
const Vector3& GetAngularMomentum() const;
const Matrix3& GetROrientation () const;
const Vector3& GetLinearVelocity() const;
const Vector3& GetAngularVelocity() const;

```

```

typedef Vector3 (*Function)
(
    float,          // time of application
    float,          // mass
    const Vector3&, // position
    const Quaternion&, // orientation
    const Vector3&, // linear momentum
    const Vector3&, // angular momentum
    const Matrix3&, // orientation
    const Vector3&, // linear velocity
    const Vector3&  // angular velocity
);

```

```

Function Force;
Function Torque;

void Update (float fT,float fDT);
protected:
    float m_fMass,m_fInvMass;
    Matrix3 m_omatrixInertia,m_omatrixInvInertia;

    Vector3 m_vPos;
    Vector3 m_vLinMom;
    Vector3 m_vAngMom;
    Quaternion m_vQOrient;

    Matrix3 m_omatrixROrient;
    Vector3 m_vLinVel;
    Vector3 m_vAngVel;
};

```

จะเห็นว่ามีค่าให้กับตัวแปรต่างๆ ให้กับวัตถุซึ่งค่าบางอย่างจำเป็นต้องกำหนดให้กับวัตถุไม่มีการตั้งค่าเริ่มต้นอาจทำให้ไม่สามารถนำไปคำนวณการเคลื่อนที่ของวัตถุได้ และนอกจากเราสามารถตั้งค่าเหล่านั้นได้แล้วเรายังสามารถนำค่าต่างๆออกมาใช้ได้ ส่วนในการใช้งาน ต้องมีการสร้าง ฟังก์ชัน Force และ ฟังก์ชัน Torque ให้กับ RigidBody เพื่อนำไปใช้ในการคำนวณตำแหน่งและตัวแปรต่างๆ ณ. เวลาถัดไป โดยที่หนึ่ง Rigid Body จะแทนแค่ หนึ่งวัตถุเท่านั้น

3.4.3. ส่วนคำนวณปฏิสัมพันธ์หลังการชน (Collision Response)

ในส่วนนี้จะใช้ควบคู่กับคลาส Contact ซึ่งคลาสนี้จะทำหน้าที่เก็บข้อมูลหลังการชนกันของวัตถุเพื่อนำไปเป็น Input ให้กับคลาส Collision Response ไปคำนวณแรงและทิศทางรวมไปถึงความเร็วหลังจากที่ชนกัน รูปแบบของคลาส Contact จะเป็นดังนี้

```

Class Contact
{
Public:
    RigidBody A;    //Rigid Body A at Collision
    RigidBody B;    //Rigid Body B at Collision
    Vector3 N;      // Normal of face Rigid Body A
    Vector3 PA;     //Contact Point A;

```

```
Vector3 PB;    //Contact Point B;

}
```

จะเห็นได้ว่าการเก็บค่าของ Rigidbody ของวัตถุ แรก และเก็บค่าของ Rigidbody ของวัตถุที่สอง และเก็บค่าของ Normal หน้าที่เกิดการชนกันของวัตถุ และเก็บจุดที่ชนกัน โดยทั้งหมดนี้ผู้ใช้งานต้องเป็นคนป้อนค่าเอง ซึ่งค่าเหล่านี้จะเป็นอินพุทของ CollisionResponse โดยมีรูปแบบของคลาสดังนี้

```
class CollisionResponse
{
public:

    CollisionResponse(std::vector<Contact>& oContact);

    void ComputePreimpulseVelocity (float* rgfPreRelVel);
    void ComputeImpulseMagnitude (float* rgfPreRelVel, float* rgfImpulseMag);
    void DoImpulse (float* fImpulseMag);

    void Compute ();
    void SetContact(std::vector<Contact>& roContact);
    void AddContact(Contact roContact);
    std::vector<Contact> GetContact();
    int GetiNumContact();
    void Clear();
    void SetRestitution(float fRestitution);
    float GetRestitution() const;
    std::vector<Contact>& m_roContact;

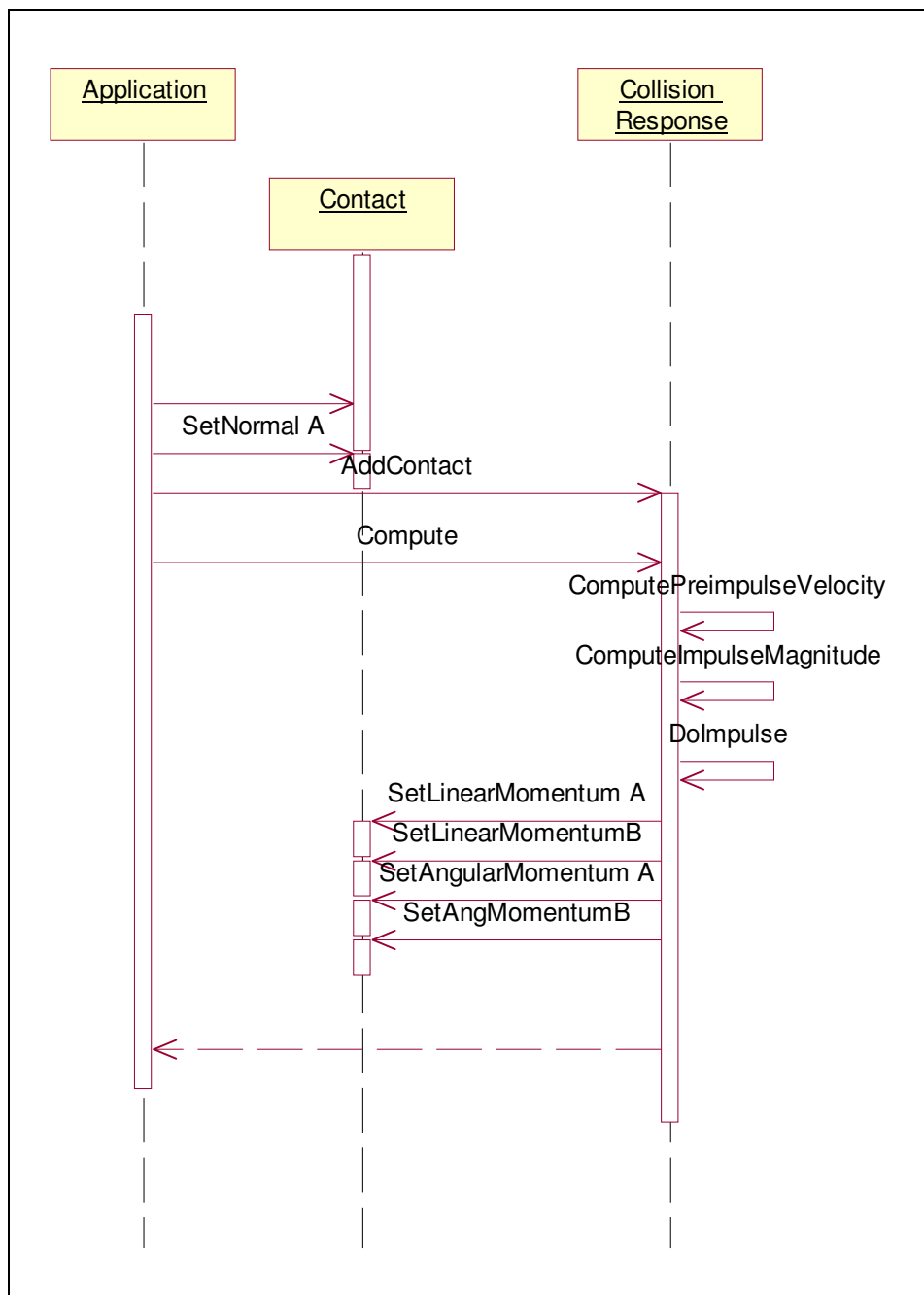
private:
    float m_fRestitution;
    int m_iNumContacts;
};
```

ในการคำนวณจะเรียกผ่านฟังก์ชัน Compute โดยในฟังก์ชันนี้จะทำการ อัปเดตค่า ทิศทางและความเร็วให้กับ Rigidbody ทั้งสองโดยการเรียก ฟังก์ชันอีกสามตัวคือ ComputePreimpulseVelocity , ComputeImpulseMagnitude , DoImpulse ซึ่งในแต่ละตัว ทำหน้าที่ดังนี้

ComputePreimpulseVelocity – ทำหน้าที่คำนวณความเร็วลัพธ์ก่อนการชนกัน
ในแนว normal เพื่อดูว่าบอลที่ชนกันนั้นมีโอกาสที่จะเคลื่อนที่ตามกันหรือไม่

ComputeImpulseMagnitude - ทำหน้าที่ คำนวณ impulse ที่เกิดขึ้นระหว่างสอง
วัตถุ

DoImpulse – ทำการนำ impulse ที่ได้มา โมเมนตัมให้กับแต่ละวัตถุ
โดยจะเป็นไปตามรูปที่ 3.44



รูปที่ 3.44 แสดงลำดับการเรียกใช้งาน Collision Response

เมื่อได้ตามนี้จะทำให้เราสามารถหาได้ว่าหลังจากชนวัตถุจะไปในทิศทางใด ซึ่งถ้าต้องการผลลัพธ์ที่ถูกต้องจะต้องมีการป้อนตัวแปรให้ถูกต้องตามไปด้วย

3.5 ทฤษฎีที่เกี่ยวข้องกับ Physically Base Simulation

3.5.1 ส่วนตรวจสอบการชนกัน

ในส่วนนี้เป็นการนำสมการทางคณิตศาสตร์มาคำนวณหาการชนกันของวัตถุต่างๆ ซึ่งมีความซับซ้อนอยู่บ้าง โดยจะขอยกตัวอย่างขึ้นมาบางส่วนดังต่อไปนี้

3.5.1.1. การตรวจสอบการชนระหว่างเส้นตรงกับสามเหลี่ยม

ในส่วนของการตรวจจับการชนระหว่างเส้นตรงกับสามเหลี่ยมนั้น สมการที่นำมาแสดงเส้นตรงคือ $X(t) = P + tD$ โดย D คือเวกเตอร์หนึ่งหน่วยแสดงทิศทางของเส้นตรง, P เป็นจุดเริ่มของเส้นตรง, t เป็นจำนวนเต็มใดๆ โดยสามารถบอกประเภทของเส้นตรงได้จากค่าของ t ซึ่ง ถ้าเป็น เรย์ ค่า t จะมากกว่าหรือเท่ากับ 0 แต่ถ้าเป็นส่วนของเส้นตรง ค่า t จะเป็นสมาชิกของ t เริ่มต้นและ t สุดท้ายของส่วนของเส้นตรง สำหรับสามเหลี่ยมที่ประกอบไปด้วยจุด V_0, V_1, V_2 จะถูกแสดงอยู่ในรูป barycentric coordinate ตามสมการที่ (3.17)

$$b_0V_0 + b_1V_1 + b_2V_2 = V_0 + b_1E_1 + b_2E_2 \quad (3.17)$$

โดยที่ $b_0 + b_1 + b_2 = 1$ และ $E_1 = V_1 - V_0, E_2 = V_2 - V_0$ และ $0 \leq b_1, 0 \leq b_2$ แต่ $b_1 + b_2 \leq 1$ เมื่อนำสมการทั้งสองมารวมกันเพื่อจะหาจุดตัด และกำหนดให้ $Q = P - V_0$ จะได้สมการที่ (3.18)

$$Q + tD = b_1E_1 + b_2E_2 \quad (3.18)$$

ซึ่งในสมการ(2.4)นี้มีค่าสามค่าที่เราไม่ทราบคือค่าของ t, b_1 และ b_2 ดังนั้นจึงต้องทำการแก้สมการเพื่อหาค่าทั้งสามทีละค่าโดยเริ่มจาก กำหนดให้ $N = E_1 \times E_2$ จากนั้นทำการครอส E_2 กับด้านขวาของทุกเทอม ในสมการที่ (3.18) จะได้สมการที่ (3.19)

$$Q \times E_2 + tD \times E_2 = b_1E_1 \times E_2 + b_2E_2 \times E_2 \quad (3.19)$$

จัดรูปสมการที่ (2.5) ได้ เนื่องจาก $E_2 \times E_2 = 0$ ดังนั้นจะได้สมการที่ (3.20)

$$Q \times E_2 + tD \times E_2 = b_1N \quad (3.20)$$

จากนั้นทำการ คอท สมการ(3.20) ด้วยค่า D จะได้ สมการที่ (3.21)

$$D \cdot Q \times E_2 + tD \cdot D \times E_2 = b_1 D \cdot N \quad (3.21)$$

จัดรูปสมการที่ (3.21) ใหม่ได้สมการที่ (3.22)

$$D \cdot Q \times E_2 = b_1 D \cdot N \quad (3.22)$$

เนื่องจากว่า เส้นตรงจะตัดกับสามเหลี่ยมได้นั้นเส้นตรงต้องไม่ขนาน กับ สามเหลี่ยม แปลว่า $N \cdot D \neq 0$ และจากสมการที่ (3.22) จะได้ค่าของ สมการที่ (3.23)

$$b_1 = \frac{D \cdot Q \times E_2}{D \cdot N} \quad (3.23)$$

ทำ เหมือนเดิมอีกครั้งตั้งแต่สมการที่ (3.19)-(3.22) แต่คราวนี้ทำการครอส E_1 กับ ด้านซ้ายของทุกเทอมในสมการ (3.19) เสร็จแล้วทำการดอทผลลัพธ์ที่ได้ด้วย D ก็จะได้ เป็นสมการที่ (3.24)

$$b_2 = \frac{D \cdot E_1 \times Q}{D \cdot N} \quad (3.24)$$

สุดท้ายจะหา ค่า t โดยการนำเวกเตอร์ N ดอทเข้าไปกับทุกเทอมในสมการที่ (3.19) เมื่อเวกเตอร์ E_1 และ E_2 ดอทกับเวกเตอร์ N แล้วจะได้ เท่ากับ 0 เพราะว่า ทั้ง E_1 และ E_2 ตั้งฉากกับ N นั้นเองจะได้ ดังสมการที่ (3.25)

$$t = \frac{-Q \cdot N}{D \cdot N} \quad (3.25)$$

ค่าที่ได้จากสมการที่ (3.23) , (3.24) , (3.25) เป็นตัวแปลของจุดที่ชนกันระหว่าง เส้นตรงและสามเหลี่ยม ซึ่งถ้าเป็น เรย์ จะตรวจสอบจากค่า t โดย t จะต้องมากกว่าเท่ากับ 0 แต่ถ้าเป็นส่วนของเส้นตรง t จะต้องมีการระหว่าง t เริ่มต้นและ t สุดท้ายของส่วนของ เส้นตรง ถ้าไม่ตรงตามเงื่อนไขดังกล่าวก็แสดงว่าไม่เกิดการชน แต่ถ้าตรงตามเงื่อนไขจุด ที่ชนจะเป็นจุดเดียวกับเส้นตรงนั่นเอง

3.5.1.2. การตรวจสอบการชนระหว่างเส้นตรงกับทรงกลม

ในการตรวจสอบนั้นได้ใช้ สมการเส้นตรง คือ $X(t) = P + tD$ โดย D คือเวกเตอร์ หนึ่งหน่วยแสดงทิศทางของเส้นตรง , P เป็นจุดเริ่มของเส้นตรง , t เป็นจำนวนเต็มใดๆ

โดยสามารถบอกประเภทของเส้นตรงได้จากค่าของ t ซึ่ง ถ้าเป็น เรย์ ค่า t จะมากกว่าหรือเท่ากับ 0 แต่ถ้าเป็นส่วนหนึ่งของเส้นตรง ค่า t จะเป็นสมาชิกของ t เริ่มต้นและ t สุดท้ายของ ส่วนของเส้นตรง และนำสมการอีกสมการหนึ่งคือ สมการวงกลม คือ $|X - C| = r$ โดย ที่ C คือจุดศูนย์กลางของทรงกลมและมีรัศมีเท่ากับ r จากนั้นเราจะทำการคำนวณหาจุดที่ เส้นตัดกับทรงกลมได้จากการนำสมการเส้นตรงเข้ามาแทนที่ X ในสมการทรงกลมจะได้ ดังสมการที่ 3.26

$$\begin{aligned} 0 &= |X - C|^2 - r^2 = |tD + P - C|^2 - r^2 = |tD + \Delta|^2 - r^2 \\ &= t^2 + 2D \bullet \Delta t + |\Delta|^2 - r^2 \end{aligned} \quad (3.26)$$

เมื่อทำการแก้สมการหาค่าของ t ที่จะได้ดังสมการที่ 3.27

$$t = -D \bullet \Delta \pm \sqrt{(D \bullet \Delta)^2 - (|\Delta|^2 - r^2)} \quad (3.27)$$

จากสมการที่ 3.27 ทำการกำหนดตัวแปล $A = \sqrt{(D \bullet \Delta)^2 - (|\Delta|^2 - r^2)}$ ซึ่งถ้า A มีค่ามากกว่าศูนย์แสดงว่าจะมีจุดของเส้นที่ตัดกับทรงกลมสองจุด แต่ถ้า A เท่ากับ 0 จะมี จุดตัดกับทรงกลมเพียง 1 จุด และถ้าหาก A น้อยกว่าศูนย์ ซึ่งจะทำให้ผลลัพธ์ที่ได้ไม่ใช่ จำนวนจริง แสดงว่าเส้นตรงไม่ตัดกับทรงกลม เราจึงสรุปได้ว่า ถ้า A เท่ากับหรือมากกว่า 0 แสดงว่าเส้นตรงตัดกับทรงกลม แต่ถ้าน้อยกว่า 0 แสดงว่าเส้นตรงไม่ตัดกับทรงกลม

3.5.1.3. การตรวจสอบการชนระหว่างเส้นตรงกับกล่อง

ในการตรวจสอบนี้จะใช้การประมวลผลในแบบสองมิติ โดยมีสมการดังนี้

$$\begin{aligned} |U_0 \bullet D \times \Delta| &> e_1 |D \bullet U_2| + e_2 |D \bullet U_1| \\ |U_1 \bullet D \times \Delta| &> e_0 |D \bullet U_2| + e_2 |D \bullet U_0| \\ |U_2 \bullet D \times \Delta| &> e_0 |D \bullet U_1| + e_1 |D \bullet U_0| \end{aligned} \quad (3.28)$$

โดยที่ $\Delta = P - C$, D เท่ากับทิศทางของเส้นตรง U เท่ากับแกนของสี่เหลี่ยมซึ่งมี ทั้งหมดสามแกนตามลำดับ e เท่ากับขนาดของแต่ละด้านตามแนวแกนของสี่เหลี่ยม ถ้า ตรงตามเงื่อนไขใดเงื่อนไขหนึ่งตามสมการที่ แสดงว่าเส้นตรงไม่ตัดกับกล่อง แต่ถ้าไม่ ตรงทุกเงื่อนไขแสดงว่าเส้นตรงตัดกับกล่อง

ในตัวอย่างทั้งสามนี้เป็นตัวอย่างที่ยกมาให้เห็นว่าในการหาการชนกันของวัตถุนั้น ได้อาศัยสมการทางคณิตศาสตร์เข้ามาช่วยในการคำนวณ รวมไปถึงรูปทรงเลขาคณิตเข้ามาช่วย

3.5.1.4. การตรวจสอบการชนกันระหว่าง ทรงกลมกับทรงกลม

ในการตรวจสอบนั้นสามารถหาได้โดยอาศัยสมการ $|X - C| = r$ โดยที่ C คือจุดศูนย์กลางของทรงกลมและมีรัศมีเท่ากับ r จากนั้นทำการแทนค่า X ด้วย $C + r$ ของอีกทรงกลมหนึ่ง เราจะได้สมการที่ 3.29

$$C_1 - C_2 = r_1 + r_2 \quad (3.29)$$

จากสมการที่ทำการยกกำลังทั้งสองข้างเพื่อให้ค่าที่ได้ออกมาเป็นบวกเท่านั้น จะได้เป็นสมการที่

$$(C_1 - C_2)^2 = (r_1 + r_2)^2 \quad (3.30)$$

ซึ่งจากสมการที่ ถ้า $(C_1 - C_2)^2$ น้อยกว่าหรือเท่ากับ $(r_1 + r_2)^2$ แสดงว่าทรงกลมทั้งสอง Intersect กัน แต่ถ้ามากกว่าแสดงว่าไม่ Intersect กัน

3.5.2. ส่วนการจำลองการเคลื่อนที่ (Rigid body simulation)

การจำลองการเคลื่อนที่ของวัตถุแข็งนั้นจะใช้การเลียนแบบการเคลื่อนที่ตาม สมการฟิสิกส์ โดยดูจากความสัมพันธ์ระหว่างสมการฟิสิกส์ ซึ่งมีอยู่สองประเภทด้วยกันคือการเคลื่อนที่เชิงเส้น, การเคลื่อนที่เชิงมุม

3.5.2.1. การเคลื่อนที่เชิงเส้น

ในการเคลื่อนที่เชิงเส้นนั้นจะนำสมการของ โมเมนตัมเชิงเส้นมาใช้ตามสมการที่ (3.31)

$$P(t) = mV(t) \quad (3.31)$$

โดยที่ P = โมเมนตัมของวัตถุ ณ. เวลานั้น

m = มวลของวัตถุ

V = ความเร็วของวัตถุ ณ. เวลานั้น

เนื่องจาก ถ้าเรารู้โมเมนตัมและรู้มวลเราจะหาความเร็วของวัตถุได้ และจากสมการที่ (3.31) ถ้านำมาหาอนุพันธ์จะได้เป็นแรงที่กระทำกับวัตถุ

$$\dot{P} = F \quad (3.32)$$

จากสมการที่ (3.32) เรารู้แรงเราจะสามารถหาความเร่งของวัตถุ จากความคิดดังกล่าว จะได้ตัวที่มีผลทำให้วัตถุเคลื่อนที่ในเชิงเส้น คือ ค่าความเร็ว และแรงที่กระทำกับวัตถุ

3.5.2.2. การเคลื่อนที่เชิงมุม

ในการเคลื่อนที่เชิงมุมจะอาศัยสมการของ โมเมนตัมเชิงมุมมาใช้ตามสมการที่ 3.33

$$L(t) = I(t)\omega(t) \quad (3.33)$$

โดยที่ L = โมเมนตัมเชิงมุม

I = Inertia Tensor

ω = ความเร็วเชิงมุม

เมื่อนำสมการที่ (3.33) มาหาอนุพันธ์จะได้ ค่าของความเร่งเชิงมุมตามสมการ(3.34)

$$\dot{L}(t) = \tau(t) \quad (3.34)$$

ซึ่ง Inertia Tensor $I(t)$ ในสมการที่ (3.33) หมายถึงปริมาณที่บอกสมบัติในการด้านการเปลี่ยนสภาพ การหมุนของวัตถุ สามารถคำนวณได้จาก สมการที่ (3.35)

$$I(t) = \sum \begin{pmatrix} m_i(r_{iy}^{'2} + r_{iz}^{'2}) & -m_i r_{ix}' r_{iy}' & -m_i r_{ix}' r_{iz}' \\ -m_i r_{iy}' r_{ix}' & m_i(r_{ix}^{'2} + r_{iz}^{'2}) & -m_i r_{iy}' r_{iz}' \\ -m_i r_{iz}' r_{ix}' & -m_i r_{iz}' r_{iy}' & m_i(r_{ix}^{'2} + r_{iy}^{'2}) \end{pmatrix} \quad (3.35)$$

ในความเป็นจริง $I(t)$ ไม่ใช่ค่าคงที่ ในการเปลี่ยนแปลงเกิดขึ้นตลอดเวลาเนื่องจากการหมุนของวัตถุ ดังนั้นเราจึงต้องหาความสัมพันธ์ระหว่าง I ที่เขียนอยู่ในรูปของ Body space และ $I(t)$ ซึ่งได้ความสัมพันธ์ดังสมการที่ (3.36) และ (3.37)

$$I(t) = R(t)I_{body}R(t)^T \quad (3.36)$$

$$I^{-1}(t) = (R(t)I_{body}R(t)^T)^{-1} = R(t)I_{body}^{-1}R(t)^T \quad (3.37)$$

ถึงจุดนี้เราสามารถสรุปสถานะของวัตถุให้อยู่ในรูปของเวกเตอร์ได้โดยทำการเก็บค่าที่แสดงสถานะของวัตถุดังสมการที่ (3.38)

$$Y(t) = \begin{pmatrix} x(t) \\ R(t) \\ P(t) \\ L(t) \end{pmatrix} \quad (3.38)$$

เวกเตอร์นี้ประกอบไปด้วยตำแหน่งวัตถุ การหมุนวัตถุ Momentum เชิงเส้น Momentum เชิงมุม ซึ่ง Momentum เชิงเส้น Momentum เชิงมุม จะแสดงข้อมูลด้านความเร็ว ซึ่งมีความสัมพันธ์ตามสมการ ที่ (3.39) (3.40) (3.41)

$$v(t) = \frac{P(t)}{M} \quad (3.39)$$

$$\omega(t) = I(t)^{-1} L(t) \quad (3.40)$$

อนุพันธ์ของ $Y(t)$ ในสมการที่ (3.38) จะอยู่ในรูป

$$\frac{d}{dt} Y(t) = \frac{d}{dt} \begin{pmatrix} x(t) \\ R(t) \\ P(t) \\ L(t) \end{pmatrix} = \begin{pmatrix} v(t) \\ \omega(t) * R(t) \\ F(t) \\ \tau(t) \end{pmatrix} \quad (3.41)$$

ค่าของอนุพันธ์ของ $Y(t)$ ในสมการที่ (3.41) จะสามารถบอกตำแหน่งและความสัมพันธ์ของการเคลื่อนที่ ณ เวลาต่อไปทำให้เราสามารถทำนายการเคลื่อนที่ของวัตถุได้

3.2.5.3. ส่วนคำนวณปฏิสัมพันธ์หลังการชน (Collision Response)

สามารถคำนวณทิศทางของความเร็วหลังจากการชนกันได้จากสมการ

$$\hat{n} \bullet (\dot{p}_a^+ - \dot{p}_b^+) = -\varepsilon (\hat{n} \bullet (\dot{p}_a^- - \dot{p}_b^-)) \quad (3.42)$$

โดยที่ $\hat{n}$ คือ normal ของวัตถุ b และ $\dot{p}_a^+$ คือความเร็วที่เกิดหลังจากการชนของวัตถุ a , $\dot{p}_b^+$ คือความเร็วที่เกิดหลังจากการชนของวัตถุ b , ε เท่ากับค่าคงที่ในการสูญเสียแรงในการชน , $\dot{p}_a^-$ คือความเร็วก่อนเกิดการชนของวัตถุ a , $\dot{p}_b^-$ คือความเร็วก่อนเกิดการชนของวัตถุ b ซึ่ง รายละเอียดที่มาของสมการข้างบนได้มาจากการคำนวณหาค่าของ v_{rel} (velocity relative) โดยค่านี้คำนวณมาจากสมการที่ 3.43

$$\dot{p}_a(t_0) = v_a(t_0) + \omega_a(t_0) \times (p_a(t_0) - x_a(t_0)) \quad (3.43)$$

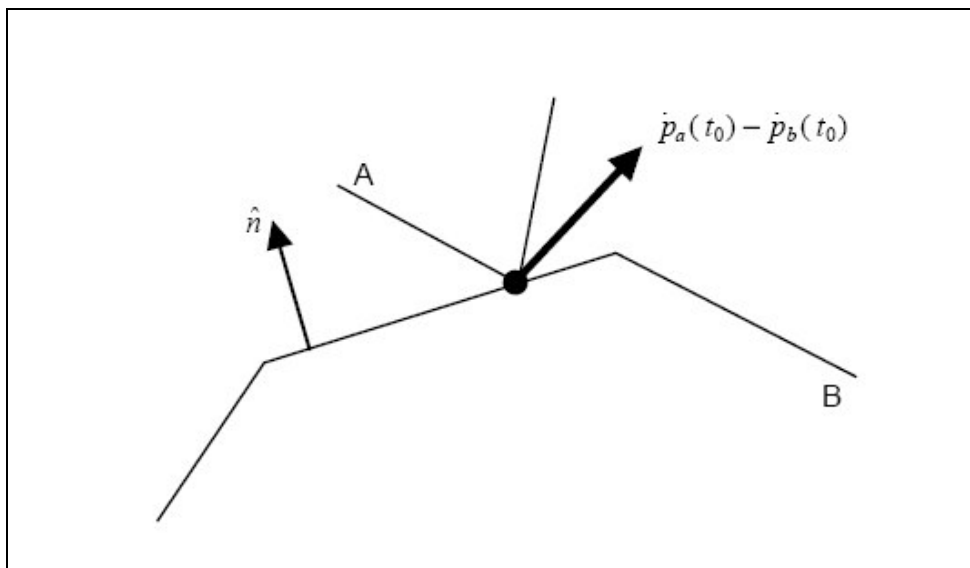
ซึ่งเป็นค่าความเร็ว ณ จุดที่ชนกันของวัตถุ a ก่อนการชน โดยที่ v_a คือความเร็วเชิงเส้นของวัตถุ a และ ω_a คือความเร็วเชิงมุมของวัตถุ a คอสมกับ จุดที่ชนกันของวัตถุ (p_a) ลบกับตำแหน่งจุดศูนย์กลางของวัตถุ a (x_a) ณ เวลาเดียวกันคือ (t_0) และสมการที่

$$\dot{p}_b(t_0) = v_b(t_0) + \omega_b(t_0) \times (p_b(t_0) - x_b(t_0)) \quad (3.44)$$

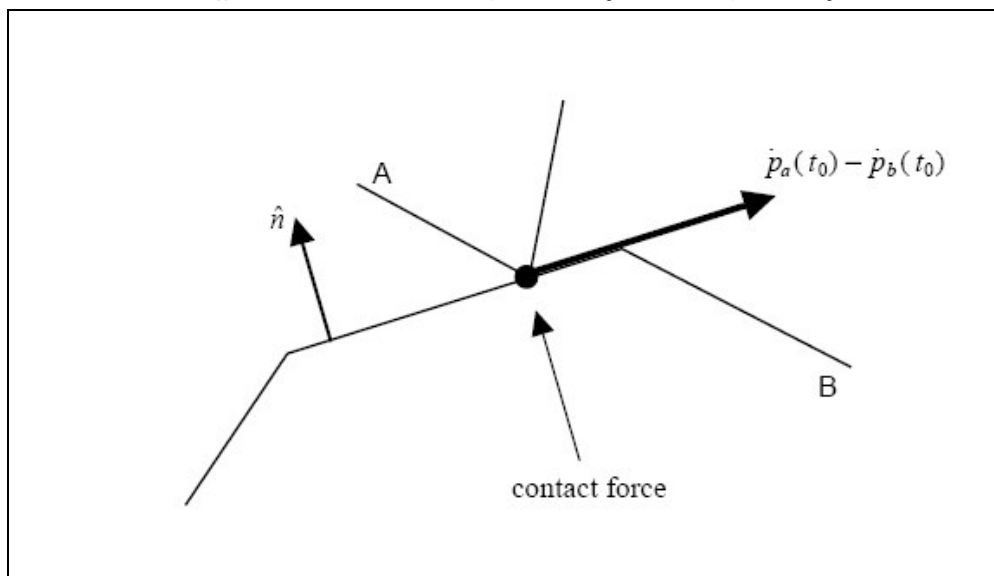
ซึ่งเป็นค่าความเร็ว ณ จุดที่ชนกันของวัตถุ b ก่อนการชน โดยที่ v_b คือความเร็วเชิงเส้นของวัตถุ b และ ω_b คือความเร็วเชิงมุมของวัตถุ b คอสมกับ จุดที่ชนกันของวัตถุ (p_b) ลบกับตำแหน่งจุดศูนย์กลางของวัตถุ b (x_b) ณ เวลาเดียวกันคือ (t_0) จากนั้นนำสมการทั้งสองมาหา v_{rel} (velocity relative) ซึ่งเป็นไปตามสมการที่

$$v_{rel} = \hat{n}(t_0) \bullet (\dot{p}_a(t_0) - \dot{p}_b(t_0)) \quad (3.45)$$

โดยที่ v_{rel} คือ (velocity relative) $\hat{n}$ คือ normal หนึ่งหน่วย $\dot{p}_a$ คือค่าที่ได้จากสมการ ที่ และ $\dot{p}_a$ คือค่าที่ได้จากสมการ ที่ ที่เราต้องทำการหา v_{rel} ก็เพื่อต้องการรู้ว่าวัตถุกำลังเคลื่อนที่ในทิศทางใด ซึ่งถ้า v_{rel} มากกว่าศูนย์แสดงว่าความเร็วนี้มีทิศทางเดียวกับ normal สามารถดูได้จากรูปที่ 3.45

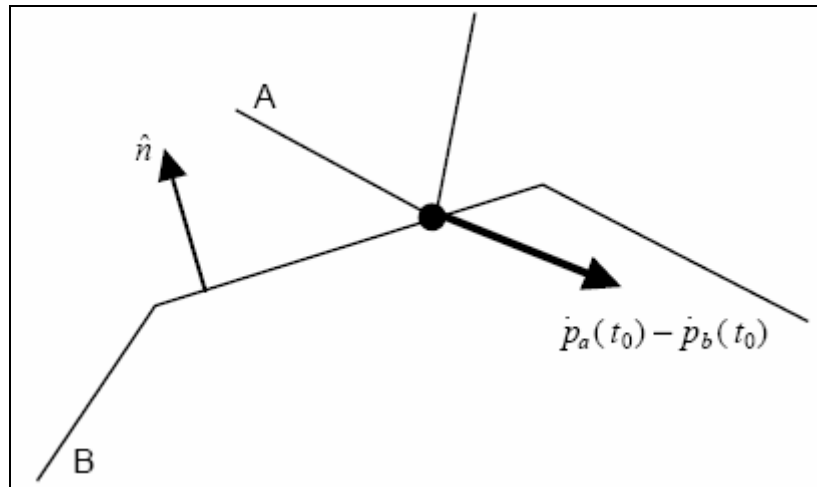


รูปที่ 3.45 แสดงความเร็วที่สัมพันธ์กันของจุดที่สัมผัสกันเมื่อ v_{rel} เป็นบวก
แต่ถ้า v_{rel} เท่ากับ 0 แสดงว่าวัตถุนั้นไถลอยู่บนอีกวัตถุหนึ่งดังรูปที่ 3.46



รูปที่ 3.46 แสดงความเร็วที่สัมพันธ์กันของจุดที่สัมผัสกันเมื่อ v_{rel} เท่ากับ ศูนย์

แต่ถ้า v_{rel} ติดลบแสดงว่าวัตถุนั้นเคลื่อนที่เข้าไปในวัตถุอีกวัตถุหนึ่งดังตัวอย่างในรูปที่ 3.47



รูปที่ 3.47 แสดงความเร็วที่สัมพันธ์กันของจุดที่สัมผัสกันเมื่อ v_{rel} ติดลบ

โดย impulse ที่เกิดจะเป็นแรงที่กระทำกับจุดสัมผัส ณ. เวลาที่น้อยมากซึ่งจะได้สมการดังสมการที่ 3.46

$$F\Delta t = J \quad (3.46)$$

J คือ impulse , F คือแรงที่กระทำกับจุด Δt คือเวลาที่น้อยมากเกือบเข้าใกล้ ศูนย์ ตัวอย่างเช่น ถ้าเราต้องการนำแรง impulse ที่ได้ไปกระทำกับวัตถุมวล M เราจะได้ความเร็วของมวลนั้นเท่ากับสมการที่

$$\Delta v = \frac{J}{M} \quad (3.47)$$

จากสมการที่ คือความเร็วที่เปลี่ยนไปจะเท่ากับ impulse หารด้วย มวล ซึ่งเราสามารถสรุปได้ว่า J คือโมเมนตัมที่เปลี่ยนไปของมวล แต่ถ้าเราต้องการหาทอร์กที่เกิดจาก impulse ด้วยก็สามารถคำนวณได้จาก สมการที่

$$\tau_{impulse} = (p - x(t)) \times J \quad (3.48)$$

$\tau_{impulse}$ คือ ทอร์กที่เกิดจาก impulse

P คือ ตำแหน่งที่ชนกัน

X คือ จุดศูนย์กลางวัตถุ

J คือ impulse ที่เกิดขึ้น

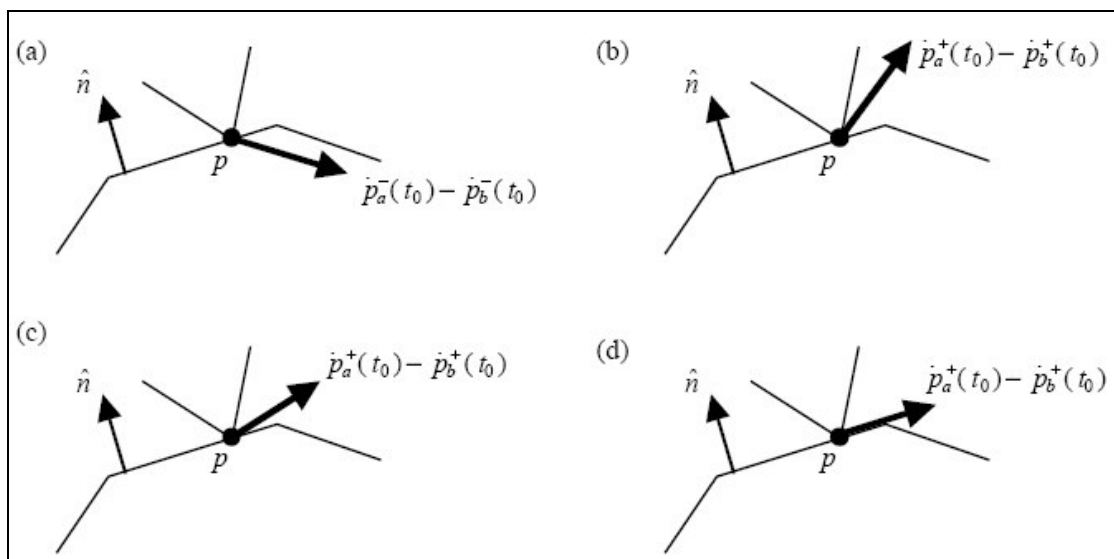
ดังนั้นถ้าเราสามารถรู้ความเร็วและทิศทางหลังการชนได้จากสมการที่

$$v_{rel}^+ = -\epsilon v_{rel}^- \quad (3.49)$$

โดยที่ v_{rel}^+ คือความเร็วและทิศทางหลังการชน

v_{rel}^- คือ ความเร็วและทิศทางก่อนการชน

ϵ คือค่าการสูญเสียที่เกิดในการชน โดยค่านี้ จะเป็นตัวกำหนดว่ารูปแบบของแรงหลังการชนนั้นจะออกมาในลักษณะใด โดยสามารถดูได้จากรูปที่ 3.48



รูปที่ 3.48 A แสดงทิศทางของความเร็วก่อนผ่านการคำนวณ impulse

B แสดง ทิศทางหลังจากผ่านการคำนวณ impulse โดยมี $\varepsilon = 1$

C แสดง ทิศทางหลังจากผ่านการคำนวณ impulse โดยมี ε ระหว่าง 0 ถึง 1

D แสดง ทิศทางหลังจากผ่านการคำนวณ impulse โดยมี $\varepsilon = 0$

จากทั้งหมดนี้สามารถสรุปเป็น สูตรสมการที่ และเมื่อผ่านการคำนวณ impulse แล้วก็ทำการ อัปเดตค่าให้กับวัตถุที่ชนกันทั้งค่า ของ โมเมนตัมเชิงเส้น และโมเมนตัมเชิงมุม ทำให้วัตถุมีการเคลื่อนที่เปลี่ยนแปลงไปจากเดิม

3.5 Drawing Tools

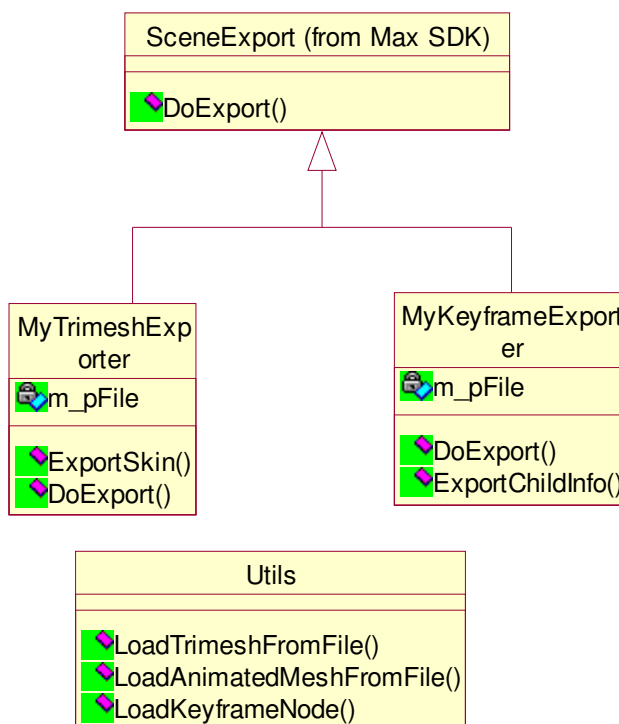
มีไว้เพื่อช่วยในการจัดรูปแบบของไฟล์ที่สามารถนำมาใช้ใน PC Engine เป็นหลักได้แก่ Terrain Height Field ที่สามารถแปลงจาก Photoshop RAW Format ไปเป็นรูปแบบที่กำหนดไว้ในส่วนของ Terrain ได้ การออกแบบไม่ซับซ้อนเขียนอยู่ในรูปแบบของ C-Style function แต่กำหนดให้เป็น static member ของคลาส Tools เพื่อสำหรับเครื่องมือที่ถูกพัฒนาในอนาคต

PC Mesh Exporter เป็น Plug-in ของ 3Ds Studio Max8 ซึ่งพัฒนาโดยใช้ MAX SDK ซึ่งมีมาพร้อมกับโปรแกรม ทำให้สามารถเข้าถึงข้อมูลของ Scene ที่ปรากฏได้โดยง่าย โดยการที่จะสร้าง Plug-in สำหรับ 3Ds Studio Max นั้นจะต้อง Derive คลาสต่างๆมาจาก MAX SDK และทำการสร้างเป็น DLL เพื่อให้ 3Ds Max นำไปใช้ต่อไป ตัว Plug-in ที่สร้างจะทำการ Export Vertices และ Indices ของ Mesh ที่กำหนดรวมไปถึง Texture Coordinate ด้วยหากต้องการ

นอกจากที่ PC Mesh Exporter จะสามารถทำการ Export Model ง่ายๆแล้ว ต้องสนับสนุนการ Export Skin ด้วย โดยให้ Export weight และ offset ของแต่ละ Vertex มาด้วยในรูปแบบที่สามารถนำไปใช้ได้ง่าย

PC Keyframe Exporter เป็น Plug-in ของ 3Ds Studio Max เช่นเดียวกันแต่ใช้สำหรับ Export Keyframe animation ของแต่ละวัตถุได้ โดยหากวัตถุใดอยู่ในลำดับชั้นเช่น Bone ก็จะทำให้การ Export ถูกให้โดยอัตโนมัติ

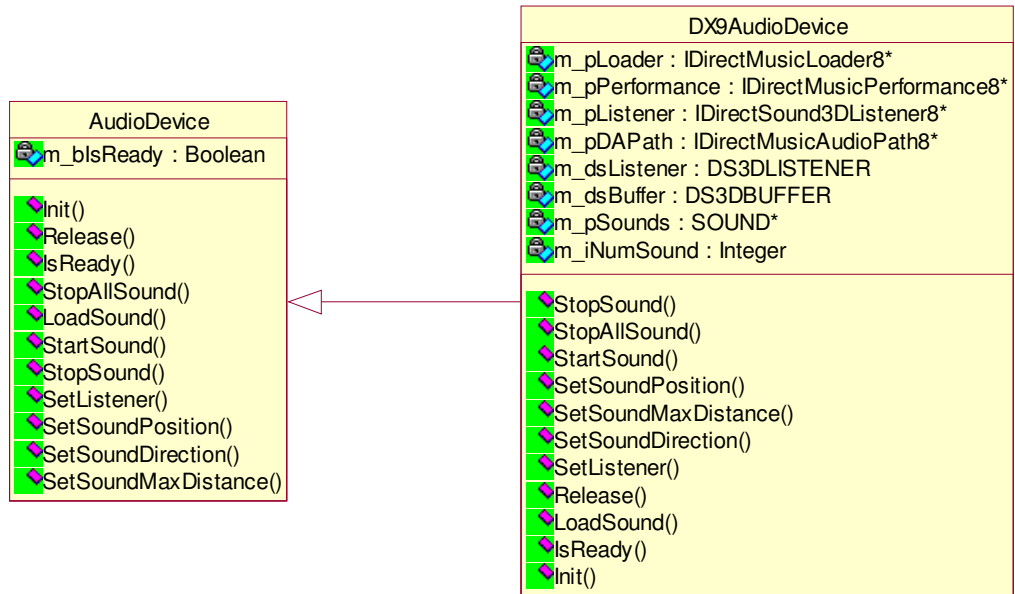
ส่วนของ Utility จะช่วยในการอ่านไฟล์รูปแบบต่างๆที่สร้างขึ้นซึ่งได้แก่ Trimesh Loader, Skin Controller Loader, Keyframe Loader การออกแบบจะง่ายไม่ซับซ้อนด้วยการสร้างแต่ละหน้าที่เป็น static member ของ Class Utils



รูปที่ 3.49 ต้นแบบ Utility และ Tools (ไม่ได้คำนึงถึงความถูกต้องในการอิมเพลเมนต์
จริงตามข้อกำหนดของ 3Ds Max SDK)

3.6. Sound Utility Module

เป็นส่วนที่ทำหน้าที่รับผิดชอบเกี่ยวกับเสียง ทั้งหมด ซึ่งในส่วนนี้จะเป็น Wrapper Class เพื่อให้ผู้พัฒนาใช้งานง่ายขึ้น โดยมี Interface ชื่อ Audio Device ซึ่ง Interface นี้สามารถ Load Sound , Start Sound (เล่นเสียง) , Stop Sound (หยุดเล่น) , Stop All Sound (หยุดเล่นทุกเสียง) Set Listener (ตั้งตำแหน่งผู้ฟัง) , Set Sound Position (ตั้งตำแหน่งของเสียง) , Set Sound Direction (ตั้งทิศทางของเสียง) Set Sound Max Distance (ตั้งระยะที่ไกลที่สุดที่จะได้ยินเสียง) ส่วนการนำไปใช้งานจะทำการ Derive Interface นี้ออกมาเป็น DirectX Audio Device หรือว่า OpenGL Audio Device ขึ้นอยู่กับผู้พัฒนา ซึ่งภายใน Audio Device และคลาสที่ Derive จะมีหน้าตา ดัง รูปที่ 3.50



รูปที่ 3.50 แสดงสมาชิกของ Audio Device และสมาชิกของคลาส DX9Audio Device ที่ Derive

ออกมา